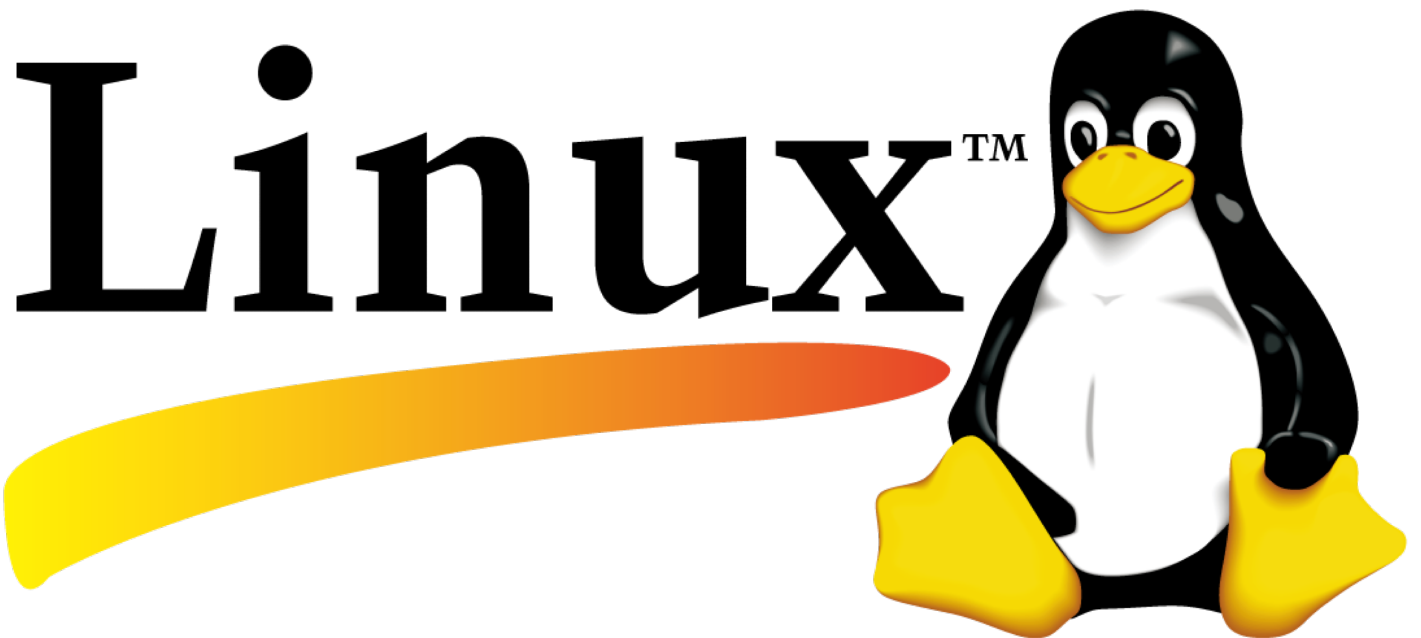




Appunti

Stacchiotti Elia

Indice

1	Generalità sul sistema Linux	2
1.1	Pacchetti software	3
1.1.1	Installazione di un pacchetto con dpkg	3
1.1.2	Rimozione di un pacchetto con dpkg	3
1.2	Ambienti Desktop Linux	4
1.3	Command Line, emulatore grafico di terminale e shell	4
1.3.1	Interazioni base con il terminale	4
1.3.2	Le Variabili	6
1.4	Navigazione nel file system	7
1.4.1	Ricerca di un file nel File system	8
1.4.2	Creazione, Spostamento ed Eliminazione di File	8
1.4.3	Il Globbing	12
1.4.4	Ricerca di dati in un file	12
1.5	Compressione e Archiviazione dei file	14
1.5.1	Strumenti per la compressione di file	14
1.5.2	Strumenti per l'archiviazione dei file	15
1.6	Redirezione di I/O	16
1.6.1	Redirigere l' Output	16
1.6.2	Redirigere l' Input	17
1.7	Directory di sistema	17
1.7.1	La directory home	18
1.7.2	La directory dev	18
1.7.3	Le directory bin	18
1.7.4	Le directory etc	19
2	Git	21
2.1	Il concetto di versione in Git	21
2.2	I Branch	25
2.3	Merge	26
3	GitHub	29
3.1	Repository remoti	29
3.2	Creare e gestire repository remoti su GitHub	29
	Fonti	34

Listati

1.1	apt install	3
1.2	apt remove	4
1.3	type	5
1.4	Esempio variabili locali, unset	6
1.5	export e env	6
1.6	Variabile PATH	7
1.7	cd e pwd	8
1.8	mkdir	8
1.9	cat	9
1.10	mv	10
1.11	rm	10
1.12	cp	11
1.13	Shell globbing	12
1.14	Contenuto dei file presenti nella directory doc	13
1.15	grep	13
1.16	gzip, zcat, bzip2, xz	15
1.17	tar	16

«««< HEAD

Generalità sul sistema Linux

Il termine Linux è spesso usato in modo improprio infatti erroneamente si crede che Linux sia un sistema operativo, in realtà non è così e **Linux è semplicemente il nome del kernel che Linus Torvalds creò nel 1991** ispirandosi a quello adottato dal sistema operativo UNIX sviluppato negli anni '70. Linux è distribuito sotto licenza GNU GPL, che **consente a chiunque di usarlo, modificarlo e dividerlo**, un **sistema operativo che incorpora il kernel Linux è chiamato distribuzione**, semplificando al massimo quello che è un sistema operativo possiamo dire che una distribuzione è un OS con kernel Linux più tanti altri software che permettono all'utente di dialogare in maniera più o meno diretta con il kernel. Con il tempo sono nate moltissime distribuzioni ciascuna delle quali risponde a particolari esigenze utente, tutte queste distribuzioni si possono raccogliere sotto 5 grandi "famiglie":

- **Debian**: una delle distribuzioni più rispettate e antiche nel panorama Linux. È la base di molte altre distribuzioni, come Ubuntu e Linux Mint. La sua filosofia **si concentra sulla stabilità, la sicurezza e la libertà del software**. Debian è nota per essere molto robusta e per il suo ciclo di rilascio lungo e prevedibile. Non è sempre la più aggiornata, ma offre una grande affidabilità, motivo per cui viene spesso utilizzata in ambienti server.
- **Red Hat**: è una delle distribuzioni più influenti e storiche, ed è la base per altre distribuzioni come CentOS (prima della sua transizione a CentOS Stream) e Fedora. Oggi, Red Hat è **focalizzata sul mercato enterprise**, offrendo Red Hat Enterprise Linux (RHEL), una distribuzione orientata alle aziende con supporto commerciale.
- **Slackware**: una delle distribuzioni più antiche e minimaliste, creata da Patrick Volkerding nel 1993. Non si basa su altre distribuzioni, ma è una distribuzione indipendente. È famosa per la sua **filosofia di minimalismo e per l'approccio "fai-da-te"**, che la rende adatta a utenti avanzati e amministratori di sistema che desiderano un controllo completo su ciò che viene installato.
- **Arch**: distribuzione indipendente, **famosa per la sua filosofia KISS (Keep It Simple, Stupid)**. Arch è **minimalista e ti offre la possibilità di costruire il sistema operativo completamente da zero**, scegliendo solo ciò che vuoi installare. Sebbene non abbia altre distribuzioni dirette da cui proviene, molte altre distribuzioni, come Manjaro, si basano su di essa.
- **Gentoo**: una distribuzione Linux avanzata che offre **un alto livello di personalizzazione**. È **basata sul principio di compilare il software direttamente dal codice sorgente**, ottimizzando ogni pacchetto per l'hardware specifico dell'utente. Anche se non ha molte "derivate" ufficiali, ci sono alcuni progetti che si basano su Gentoo o che ne adottano la filosofia di compilazione personalizzata, come Calculate Linux.

Sebbene parlare solo in questi termini di distribuzioni Linux sia abbastanza riduttivo l'obiettivo di questa parte è comunque solo fornire una panoramica andando a evidenziare i principali punti di forza delle principali.

1.1. Pacchetti software

Tutte le distribuzioni possiedono uno strumento chiamato *package manager*, questo ha la funzione di **installare tutti i vari pacchetti che servono per far funzionare una certa applicazione**. I package manager sono collegati ad una repository online da cui scaricano, su richiesta dell'utente, i pacchetti per provvedere ad installarli sulla macchina, dunque è facile capire che **cambiando package manager l'utente ha accesso ad un certo catalogo di applicazioni installabili** e quindi la scelta di quest'ultimo è importante anche se in realtà questo è qualcosa che riguarda maggiormente utenti avanzati che magari cercano applicazioni molto specifiche disponibili sono attraverso certi gestori di pacchetti. **Debian, Ubuntu e Linux Mint, utilizzano il package manager dpkg** il quale gestisce i pacchetti software indicati, in questo caso, come *pacchetti DEB*, invece Red Hat, Fedora e CentOS utilizzano il package manager *rpm* il quale gestisce i pacchetti software chiamati *pacchetti RPM*. Nel seguito si vedranno alcuni comandi molto elementari per interagire con package manager dpkg.

1.1.1. Installazione di un pacchetto con dpkg

Vogliamo ora capire come installare un applicazione sul sistema operativo, il package manager installato è dpkg, come prima cosa si deve **verificare che l'applicazione che si vuole installare non sia già presente nel sistema**, cosa che può accadere usando distribuzioni che la preinstallano, per fare questa verifica si scrive nel terminale il seguente comando e si osserva la risposta fornita dal sistema, nell'esempio il pacchetto ricercato è *figlet*:

```
$ figlet
figlet: command not found
```

nel caso la risposta del sistema sia come quella sopra riportata significa che il programma figlet non è installato, **per verificare la disponibilità del pacchetto nella repository del gestore si possono usare i comandi** `apt search <nome_pacchetto>` o `apt-cache search <nome_pacchetto>` che, a differenza del primo comando, permette di ottenere maggiori informazioni sul pacchetto disponibile, ricollegandoci all'esempio sopra digitiamo:

```
$ apt-cache search figlet
figlet - Make large character ASCII banners out of ordinary text
```

L'output sopra mostrato fornisce informazioni sul pacchetto ricercato e lo rende disponibile ad essere installato, per questa fase il **package manager richiede autorizzazioni speciali che solo l'utente chiamato root possiede e che eventualmente può concedere agli altri**, senza scendere ora nel dettaglio di questo aspetto, poichè il package manager richiede tali diritti, l'utente non root deve scrivere il **comando con il prefisso sudo che praticamente richiede all'utente la password dell'utente root prima di iniziare ad eseguire il comando**. I comandi per installare sono `apt-get install <nome_pacchetto>` che quando eseguito installa il pacchetto e mostra in output sul terminale vari aspetti tecnici dell'installazione e `apt install <nome_pacchetto>` versione più userfriendly del comando sopra in quanto installa e mostra all'utente solo eventuali errori oltre ad avere un output colorato, in riferimento all'esempio:

Listato 1.1: apt install

```
$ sudo apt-get install figlet
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
figlet
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
```

ovviamente l'output può variare da pacchetto a pacchetto ma non solo perchè ogni pacchetto richiede, per funzionare in modo adeguato, delle *dipendenze* ovvero altri pacchetti pertanto se una certa applicazione viene installata su una distribuzione particolarmente minimale è possibile che durante il processo di installazione il **package manager provveda anche all'installazione delle dipendenze collegate**.

1.1.2. Rimozione di un pacchetto con dpkg

La rimozione di un' applicazione prevede la **cancellazione sul sistema di tutti i file connessi ad essa**, i comandi usati sono `apt-get remove <nome_file>` oppure `apt remove <nome_file>` con la solita differenza

fra i due che il primo fornisce maggiori dettagli sul processo, **tali comandi tuttavia lasciano nel sistema i file di configurazione dell'applicazione** in modo che se in futuro l'utente deciderà di installare nuovamente l'applicazione questa sarà automaticamente configurata usando i file di configurazione precedentemente non rimossi, se si **vogliono eliminare anche i file di configurazione al comando classico, al posto di remove va scritto purge**, in riferimento all'esempio portato nella sezione 1.1.1:

Listato 1.2: apt remove

```
$ sudo apt-get remove figlet
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
figlet
0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.
After this operation, 741 kB disk space will be freed.
```

1.2. Ambienti Desktop Linux

Come spiegato una distribuzione Linux è sostanzialmente un insieme di software strettamente connessi al kernel Linux che permettono all'utente di utilizzare in determinati modi la macchina, *l'ambiente desktop* consiste in un gruppo ristretto di questi software che determina l'esperienza utente complessiva in termini di:

- **Gestione delle finestre:** ridimensionamento e distribuzione delle finestre nelle interfacce.
- **Pannelli e menù:** finestre che riportano dati circa l'esecuzione di applicazioni.
- **File manager:** navigazione tra cartelle e file.
- **Impostazioni di sistema:** dalla personalizzazione dell'aspetto fino alla determinazione del comportamento del sistema.

Nel mondo Linux esistono molti **ambienti desktop diversi che, insieme al package manager, definiscono in gran parte una distribuzione**; Due ambienti desktop molto famosi e presenti in molte distribuzioni sono *Gnome* e *KDE*, la prima adotta la filosofia *KISS* quindi applicazioni molto snelle e utilizza il *toolkit* *GTK*, la seconda invece ha una prospettiva più orientata alla personalizzazione utente e il *toolkit* utilizzato è *Qt*, la differenza di *toolkit* (e poi anche di altri aspetti però più secondari) può condizionare l'esperienza utente in modo abbastanza significativo se infatti l'utente utilizza applicazioni scritte con toolkit diverso da quello usato dall'ambiente desktop è necessario installare pacchetti aggiuntivi e, dal punto di vista della visualizzazione, si avverte una certa disuniformità con il resto del sistema.

1.3. Command Line, emulatore grafico di terminale e shell

Una caratteristica molto importante dell'ambiente desktop è sicuramente *l'emulatore di terminale grafico* e la *shell*, nel linguaggio comune si tende ad usare questi termini come fossero equivalenti, in realtà non è così. *L'emulatore grafico di terminale* permette di avere un'interfaccia grafica che solitamente consiste in una finestra all'interno della quale l'utente può scrivere righe di testo e dove il sistema reindirizza eventualmente output di tipo testuale, è detto emulatore perchè praticamente emula la *Command line* abbreviata con *CLI*, questa si presenta sempre come una finestra ed ha le stesse identiche funzione di un emulatore grafico di terminale, l'unica differenza sta nel fatto che la *CLI* è molto più essenziale e, anche per questo, meno personalizzabile e molto leggera, banalmente la *CLI* non permette neppure il copia e incolla del testo però il vantaggio è che risulta essere sempre accessibile dall'utente anche quando ad esempio la macchina si trova in particolari stati dove le risorse sono molto limitate tipo ripristino. La *shell* è un elemento essenziale, si tratta del programma che interpreta il testo scritto nella *CLI* o nell'*emulatore grafico di terminale* trasformando di fatto il testo in comandi che il sistema sottostante esegue, esistono vari tipi di shell ma la più comune è *Shell Bash* che sarà considerata anche per i vari esempi che seguiranno.

1.3.1. Interazioni base con il terminale

Vediamo in questa parte alcune delle interazioni base tra sistema e utente attraverso il terminale (inteso come *emulatore grafico di terminale*) o *CLI*, ricordiamo che la shell usata è *Shell Bash*. Appena si avvia

il terminale si visualizza accanto al cursore che permette l'inserimento di testo una stringa di questo tipo: `nome_utente@nome_host directory_corrente tipo_di_shell` vediamo in dettaglio in cosa consistono i vari campi:

- `nome_utente`: il nome dell'utente che esegue shell.
- `nome_host`: nome dell'host su cui viene eseguita la shell, l'host sarebbe la macchina su cui è installato il sistema.
- `directory_corrente`: **directory in cui si trova la shell durante l'esecuzione**, il simbolo `~` è usato per indicare che **la shell si trova nella directory home dell'utente corrente**.
- `tipo_di_shell`: rappresenta con un simbolo se la shell è eseguita da un utente ordinario, in tal caso il simbolo è `$`, oppure `#`, se eseguita dal superutente detto root.

In generale i comandi hanno una struttura di questo tipo: `comando [opzione(i)...] [argomento(i)...]`, dove ciò che è scritto fra parentesi quadre è facoltativo, vediamo meglio cosa richiedono questi campi:

- `nome_comando`: il nome del comando che vogliamo eseguire.
- `opzione(i)/parametro(i)`: è possibile che il comando **supporti delle opzioni o che lavori con dei parametri**, questi aspetti dipendono molto dal tipo di comando che si considera, ogni comando può essere eseguito **specificando un numero a piacere di opzioni e parametri**. Spesso un comando possiede **opzioni che sono esprimibili con formato corto o con formato lungo**, a livello di esecuzione sono identiche la differenza sta solo nel modo in cui l'opzione va dichiarata nel comando, il formato corto è quello maggiormente usato e si differenzia dall'altro perchè l'opzione è preceduta dal carattere `-` e rappresentata con una sola lettera, il formato lungo invece prevede il prefisso `--` poi varie stringhe dichiarative.
- `argomento(i)`: dati aggiuntivi richiesti dal comando, anche qui è possibile che ne siano richiesti più di uno.

La shell supporta due tipologie di comandi,

- **Interni**: sono comandi che **fanno parte della shell stessa e servono per svolgere attività all'interno della shell**, ne esistono circa 30 anche se questo numero può variare in base alla particolare shell considerata. **In genere sono chiamati comandi shell builtin.**
- **Esterni**: sono comandi **ciascuno dei quali risiede in un singolo file, in genere sono programmi o script binari** e quando si scrive nel terminale il comando (che porta lo stesso nome del file che si vuole eseguire) **la shell, non trovandolo tra i comandi interni, usa la variabile `PATH` per cercare il file**, tale variabile verrà approfondita nel seguito, ma sostanzialmente è una variabile che contiene indirizzi a directory dentro le quali la shell può cercare i comandi esterni.

Il comando `type <nome_comando>` permette di capire se il comando inserito come argomento è interno o esterno, di seguito un esempio:

Listato 1.3: type

```
$ type echo
echo is a shell builtin
$ type man
man is /usr/bin/man
```

il primo output indica che il comando `echo` è interno, mentre il secondo output indica che il comando `man` è esterno e viene indicato dove si trova. **Il quoting è una funzionalità che praticamente tutte le shell mettono a disposizione e consiste nell'utilizzo di caratteri particolari che fanno interpretare alla shell il testo scritto in determinati modi**, di seguito riporto 3 di questi caratteri:

- Virgole doppie (`"`): il testo scritto tra virgole doppie è interpretato dalla shell in modo classico quindi i caratteri speciali come il dollaro (`$`), il backslash (`\`) e il backquote (```) mantengono le funzioni classiche, tuttavia il carattere spazio non viene più interpretato come separatore di argomenti ma come carattere effettivo.
- Virgole singole (`'`): il testo scritto tra virgole singole è interpretato dalla shell come testo puro quindi tutti i caratteri speciali non sono più interpretati come funzione ma solo come testo.

- Backslash(\): i caratteri scritti di seguito al backslash fino ad un carattere spazio sono interpretati come caratteri testuali.

L'argomento *quoting* è di particolare importanza perchè tutto ciò che l'utente può fare per comunicare con il sistema è scrivere stringhe le quali vengono interpretate in un certo modo dalla shell, se l'interpretazione della stringa da parte della shell è diversa da quella che pensiamo otteniamo risultati anche molto diversi tra loro.

1.3.2. Le Variabili

Dobbiamo ricordare che la shell consiste in un programma dunque quando **avviamo un terminale o una CLI, implicitamente stiamo anche avviando la shell, in particolare stiamo aprendo una sessione shell**, durante questa sessione è possibile definire delle variabili, il motivo per cui si dovrebbe fare una cosa del genere dipende molto da che tipo di operazioni vogliamo svolgere sul sistema, ad esempio se si vogliono creare 10 file con nomi diversi potrebbe essere utile dichiarare una variabile contatore che poi dinamicamente viene decrementata, oppure una variabile che racchiude un'informazione del sistema che poi viene utilizzata per formattare stringhe e tante altre applicazioni diverse. Si possono definire durante la sessione due tipi di variabili:

- **Variabili locali:** sono variabili interne alla particolare sessione di shell, sono disponibili solo alla shell stessa e non sono utilizzabili da sottoprocessi ovvero da processi esterni alla shell (anche quelli avviati da essa). Una variabile locale si può creare nel seguente modo `<nome_variabile>=<valore_variabile>`, è importante che non siano presenti spazi nel nome, nel valore e tra il simbolo di assegnamento (=), per visualizzare sulla shell il valore della variabile si scrive il comando `echo $<nome_variabile>`, dove il carattere \$ serve per accedere al valore della variabile avente nome scritto subito dopo, per rimuovere una variabile locale si scrive `unset <nome_variabile>`, di seguito riporto un esempio molto semplice dei comandi sopra scritti:

Listato 1.4: Esempio variabili locali, unset

```
$ luce=on
$ echo $luce
on
$ luce=off
$ echo $luce
off
$ unset luce
$ echo $luce
```

- **Variabili di ambiente:** sono variabili che risultano essere visibili a tutti i sottoprocessi avviati dalla sessione shell oltre che alla shell stessa, sono molto usate per passare informazioni sul sistema ai comandi esterni che si vogliono eseguire. Molto spesso le variabili di ambiente vengono dichiarate con un nome scritto con lettere maiuscole e **l'insieme completo di tutte viene chiamato ambiente**. Le variabili ambientali **si creano usando export**, le regole per la visualizzazione e distruzione sono identiche a quelle viste per le variabili locali, è possibile ottenere una lista delle variabili di ambiente create dalla particolare shell usando il comando `env`. È possibile creare una variabile di ambiente a partire da una locale di seguito un esempio molto semplice:

Listato 1.5: export e env

```
$ export giorno=on
$ echo $giorno
on
$ unset giorno
$ echo $luce
$ macchina=rossa
$ export macchina
```

Le variabili non sono permanenti nel sistema ma restano fin tanto che non viene chiusa la sessione di shell dentro la quale sono state dichiarate, alla chiusura le variabili vengono distrutte, questo può rappresentare **un problema soprattutto per le variabili ambientali che sostanzialmente assumono sempre lo stesso valore**, per questo molte shell mettono a disposizione **un file di configurazione dentro al quale vanno dichiarate queste variabili**, in questo modo quando la shell viene avviata questa dichiara

automaticamente tutte queste variabili. Tra tutte le variabili di ambiente una delle più importanti è la **variabile PATH**, questa variabile ha come valore **un elenco di percorsi a directory separati dal carattere :** e le **directory puntate sono quelle che la shell consulta quando viene richiesta l'esecuzione di un programma invocabile come comando esterno**. Modificare la variabile PATH richiede particolare attenzione e generalmente la modifica consiste nella rimozione o aggiunta di un percorso, di seguito è riportato un esempio:

Listato 1.6: Variabile PATH

```
$ echo $PATH
/home/user/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
$ PATH=$PATH:/usr/games
$ echo $PATH
/home/user/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/usr/bin
$ PATH=/home/user/bin
$ echo $PATH
/home/user/bin
```

Una piccola nota riguarda l'ordine con cui sono scritti i percorsi della variabile PATH, questi infatti coincidono anche con l'ordine con cui la shell effettua la ricerca. Il comando `which` permette di individuare il percorso che la shell segue quando viene invocato il programma attraverso comando, la sintassi è del tipo: `which <nome_comando>`.

1.4. Navigazione nel file system

Un aspetto basilare per l'utilizzo di una qualsiasi distribuzione Linux è saper navigare all'interno del file system utilizzando riga di comando. Iniziamo con qualche nozione teorica fondamentale, nei sistemi Linux è abbastanza importante **non utilizzare caratteri speciali (spazio, virgola, slash, asterischi, cancelletto, questi elencati sono i più comuni ma per essere precisi bisognerebbe controllare quali caratteri la shell interpreta come speciali) nei nomi assegnati ai file e directory**, questo perché altrimenti la ricerca di essi potrebbe tradursi nella scrittura di un comando più complicato in quanto i caratteri speciali presenti nel nome sono interpretabili dalla shell, in linea di massima **la regola generale è usare solo lettere e numeri**. I nomi dei file spesso possiedono un suffisso finale che inizia sempre con il carattere punto, in Windows questo è chiamato estensione, **tale suffisso aiuta l'utente a ricordare che tipo di dati sono presenti nel file ma non ha alcun significato per il sistema** che infatti esegue il file comunque essendo il nome di esso solo un identificativo che non può definire quanto scritto al suo interno. Vediamo ora il **comando che permette di spostarsi tra le cartelle modificando la directory di lavoro corrente della shell, ovvero:** `cd <percorso_directory_o_file>`, il percorso al file o directory può essere assoluto o relativo:

- **Percorso assoluto:** prevede che si vadano a **specificare tutte le sottodirectory da "attraversare" per arrivare al file o directory desiderata a partire dalla directory root indicata con il carattere **.
- **Percorso relativo:** prevede che si vada ad **illustrare il percorso al file o directory desiderata rispetto all'attuale directory di lavoro in cui si trova la shell** (tale directory sarebbe quella che l'utente visualizza accanto al cursore che permette di scrivere testo nel terminale o nella CLI, negli esempi riportati in questo file solitamente viene scritto solo \$ non dando particolare importanza a quanto scritto prima ovvero nome utente, nome host e directory di lavoro, **ricorda che il carattere ~ rappresenta la directory utente il cui percorso assoluto è /home/user**). Nell'espressione dei percorsi relativi tornano molto utili i caratteri **. e ..**, **il primo serve per specificare che il percorso parte dalla directory corrente di lavoro mentre il secondo permette di spostarsi nella directory padre rispetto a quella corrente** (in pratica permette di scalare di una directory).

I percorsi relativi sono molto comodi per navigare velocemente in directory vicine mentre quelli assoluti sono utili per compiere spostamenti tra directory più distanti. Di seguito si riportano degli esempi semplici che mostrano i vari modi per raggiungere la directory report considerando una struttura del tipo: `/home/user/Documenti/test/report/immagini`, notare che inoltre in questi esempi si riporta anche la parte a sinistra del carattere \$ in quanto è rilevante in questo caso.

```
user@hostname ~ $ cd ../Documenti/test/report
```

```
user@hostname ~ $ cd /home/user/Documenti/test/report
```

```
user@hostname ~/Documenti $ cd ./test/report
```

```
user@hostname ~/Documenti/test/report/immagini $ cd ..
```

Il comando **cd**, se eseguito senza percorso sposta la directory di lavoro a quella utente (indicata con ~). Il comando **pwd** permette di ottenere il percorso assoluto della directory di lavoro, ad esempio:

Listato 1.7: cd e pwd

```
user@hostname ~/Documenti/test/report $ pwd
/home/user/Documenti/test/report
```

1.4.1. Ricerca di un file nel File system

Un altro aspetto fondamentale nella navigazione nel file system è la ricerca di file e directory, **Il comando tree mostra l'organizzazione della directory di lavoro, l'output è una rappresentazione grafica**, è possibile che l'applicazione non sia presente di default nel sistema quindi nel caso è necessario installarla. **Il comando ls <flag> serve per ottenere un elenco dei file e directory contenuti nella directory di lavoro, l'output è un elenco.** Tale comando è molto utilizzato ed è importante conoscerlo bene, per approfondimenti si rimanda alla sezione apposita, i flag disponibili sono molti ma sicuramente i più importanti sono: **-l** mostra il contenuto della directory in formato dettagliato, **-a** mostra anche i file nascosti, **-t** mostra file e directory in ordine crescente per data di modifica.

Per trovare un file o directory specifica esistono comandi appositi di seguito si parlerà dei più comuni e efficienti. Iniziamo con il comando **locate <nome_programma>**, tale comando **avvia un programma che cerca all'interno di un database e restituisce tutti i nomi dei file presenti che possiedono la stringa specificata**, notare che vengono restituiti anche i file che contengono il nome specificato come sottostringa, quest'ultimo comportamento del comando è modificabile consultando la documentazione dello stesso. Lo stesso comando **supporta espressioni regolari** per la ricerca, questo aspetto è particolarmente utile e permette di rendere la ricerca molto più mirata. Essendo che il comando consulta un database per effettuare la ricerca è possibile che non venga trovato un file recentemente aggiunto se il database non è aggiornato, l'aggiornamento di questo avviene attraverso il programma **updatedb** il quale viene eseguito periodicamente dal sistema, qualora si volesse forzare l'esecuzione e quindi l'aggiornamento è necessario essere superutente. L'altro comando utilizzabile per ricercare file usa un approccio ricorsivo, la sintassi è del tipo **find <path_start> <filtro> <espressione>**, il **path_start** specifica il punto rispetto al quale inizia la ricerca da parte del programma, il filtro serve per scegliere che tipo di caratteristica si vuole ricercare per i file, ad esempio **-name** serve per cercare un file per nome e il campo espressione contiene il valore da ricercare; Anche questo comando **supporta espressioni regolari** quindi riconosce l'utilizzo di caratteri jolly.

1.4.2. Creazione, Spostamento ed Eliminazione di File

In Linux la shell è molto usata anche per la gestione dei file, tra l'altro questo strumento in molti casi si rivela ben più utile e efficiente rispetto ad un file manager grafico, vediamo quali sono i comandi fondamentali da usare per compiere le operazioni basilari che si compiono normalmente. Per **creare una directory, che praticamente è un particolare tipo di file, si usa il comando mkdir <flag> <nome_directory>**, il particolare il comando può creare anche più directory, basta dividere il nome di esse con il carattere spazio, se non si usa il flag **-p** le eventuali sottodirectory che si vogliono creare richiedono che la directory padre esista già o che comunque sia stata creata prima. Una piccola nota riguarda il nome delle directory che Linux distingue anche se queste hanno stesso nome ma scritto con caratteri diversi (minuscolo e maiuscolo) a differenza di Windows che invece non fa questa distinzione per le directory. Di seguito è riportato un esempio molto semplice sull'utilizzo del comando **mkdir**.

Listato 1.8: mkdir

```
$ pwd
/home/snake
```

```
$ ls
documenti
$ mkdir musica musica/preferiti musica/nuovi giochi
$ ls
documenti musica giochi
$ ls musica
preferiti nuovi
```

Per la creazione di un file esistono molti comandi diversi, questo perchè ovviamente dipende anche da quello che vogliamo scriverci dentro, uno dei più semplici è il comando `touch <flag> <nome_file>` che, se usato senza flag, crea il file oppure, se già presente, ne aggiorna la data di modifica e accesso. Una volta creato il file, per modificarlo esistono vari modi, uno semplice che permette di scrivere testo nel file è quello di usare i caratteri speciali `> e >>` in combinazione al comando `echo <flag> <stringa>`, in pratica il comando `echo` permette di scrivere sul terminale del testo mentre i caratteri `> e >>` consentono di reindirizzare l'output dei comandi (quindi anche il comando `echo`) in un file, in particolare il carattere `>>` aggiunge l'output in fondo al file mentre `>` lo sovrascrive. In realtà l'utilizzo di `echo` e dei caratteri speciali `> e >>` è utile non solo per inserire testo ad un file esistente ma anche per crearlo direttamente con dentro il testo specificato. Per visualizzare il contenuto di un file come testo nel terminale si usa il comando `cat <flag> <percorso_file>`, negli utilizzi più semplici non servono opzioni da specificare. Di seguito è riportato un esempio sull'utilizzo dei comandi sopra spiegati.

Listato 1.9: cat

```
$ pwd
/home/snake/documenti
$ ls
$ touch lista-spesa appunti
$ ls
lista-spesa appunti
$ echo patate > lista-spesa
$ cat lista-spesa
patate
$ echo pomodori >> lista-spesa
$ cat lista-spesa
patate
pomodori
$ echo farina > lista-spesa
$ cat lista-spesa
farina
$ ls
lista-spesa appunti
$ echo devo andare dal medico domani alle 14:00 > note
$ ls
lista-spesa appunti note
$ cat note
devo andare dal medico domani alle 14:00
```

L'operazione di rinominazione e spostamento di file si può compiere usando il comando `mv <flag> <percorso_file_origine> <percorso_file_finale>`, il comando sposta un il/i file specificati nel primo argomento nel file directory specificato come secondo argomento (che si sottolinea che deve essere unico, cioè il comando non accetta più di una destinazione), questo funzionamento può essere sfruttato per rinominare i file (intese sia directory che tutti gli altri tipi) in quanto basta che i percorsi ai due file coincidano tranne ovviamente per il nome, per quest'ultimo utilizzo bisogna fare attenzione che il nuovo nome assegnato al file non sia già usato da un altro, in tal caso si verifica la sovrascrittura del file già esistente con quello rinominato. Il comando `mv` accetta anche i seguenti flag:

- `-i`: interattivo, se il file di destinazione usa lo stesso nome di un file già esistente chiede conferma prima di sovrascrivere.
- `-f`: forza, stessa condizione descritta per il flag `-i` però non chiede conferma ma esegue direttamente la sovrascrittura, è il comportamento di default.
- `-u`: muove solo se il file di origine è più recente rispetto al file di destinazione o se il file di destinazione non esiste.

- -v: verboso, mostra un messaggio per ogni file spostato o rinominato.

Di seguito è riportato un esempio (ricordare che l'organizzazione delle directory è stata definita nell'esempio sopra), il primo mv sposta 2 file da una directory ad un'altra, il secondo è usato per rinominare un file e il terzo per rinominare una directory

Listato 1.10: mv

```
$ pwd
/home/snake/documenti
$ ls
lista-spesa appunti
$ echo esercizio sulle derivate > esercizio1
$ ls
lista-spesa appunti esercizio1
$ mv -v esercizio1 appunti ../giochi
'esercizio1' -> '../giochi/esercizio1'
'appunti' -> '../giochi/appunti'
$ ls
lista-spesa
$ mv lista-spesa lista
$ ls
lista
$ mv ../giochi ../scuola
$ ls ..
scuola documenti musica
```

Per eliminare generici file si usa il comando `rm <flag> <percorso_al_file>`, i flag per questo comando sono di particolare importanza:

- -r: ricorsiva, questo flag permette di eliminare tutti i file e tutte le sottodirectory che si trovano nella directory passata per argomento (anche quest'ultima viene rimossa).
- -i: interattivo, se usato chiede conferma ogni volta che viene rimosso un file.
- -f: non interattivo, è il comportamento standard.

Quando il comando è usato senza il flag `-r` è possibile eliminare solo file diversi da directory e inoltre è possibile anche passare più di un file. Se viene usato il comando con il flag `-r` la portata di esso è molto più ampia e quindi solitamente bisogna fare attenzione nel suo utilizzo, infatti con il flag attivo vengono rimossi tutti i file presenti nella directory e anche nelle sottodirectory (se presenti), oltre poi alla stessa directory passata come argomento, solitamente per una questione di sicurezza nella procedura di eliminazione il flag `-r` è affiancato al flag `-i`. Esiste poi un comando appositamente utilizzabile per eliminare directory vuote ovvero `rmdir <percorso_alla_cartella>`, è possibile passare anche più di una directory. Di seguito il solito esercizio che illustra i vari utilizzi, nel primo si mostra come si eliminano più file non directory, nel secondo come si elimina una directory con dentro file e sottodirectory e nel terzo come si eliminano directory vuote, prima dell'esercizio viene mostrato l'albero della directory user (figura 1.1).

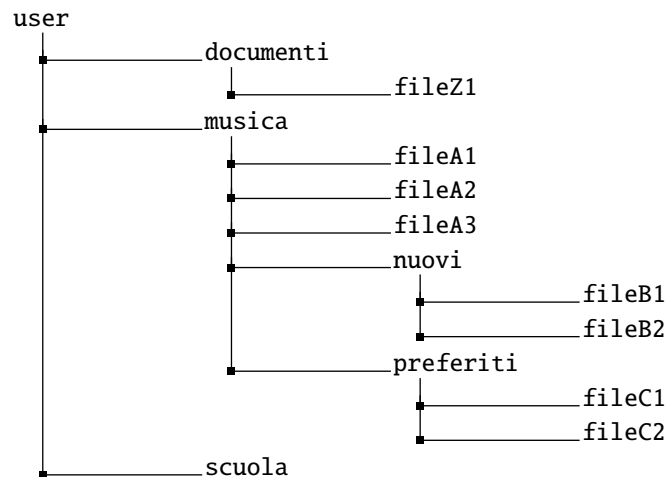
Passiamo all'esercizio effettivo:

Listato 1.11: rm

```
$ pwd
/home/user
$ ls
documenti musica scuola
$ cd musica
$ rm fileA1 fileA2 fileA3 ../preferiti/fileC1 ../preferiti/fileC2 ../documenti/fileZ1
$ ls . ../preferiti ../documenti
.:
nuovi preferiti
../preferiti:

../documenti:
$ rm -ir ../nuovi
rm: descend into directory '../nuovi'? Y
rm: remove '../nuovi/fileB1'? Y
```

Figura 1.1: Struttura interna della directory user



```

rm: remove './nuovi/fileB2'? Y
rm: remove directory './nuovi'? Y
$ ls
preferiti
$ rmdir ../documenti ../preferiti ../scuola
$ cd ..
$ ls
musica

```

Infine vediamo come si effettua la copia di un file, si utilizza il comando

`cp <flag> <percorso_file_origine> <percorso_file_destinazione>`, quando si omette il flag il comando copia i file passati per argomento nella directory di destinazione scritta come secondo argomento. Quando si ha la necessità di copiare una directory in un'altra è necessario usare il flag **-r** (unico flag disponibile per questo comando) che funziona con l'approccio ricorsivo solito, in quest'ultimo utilizzo se la directory di destinazione non esiste allora questa viene creata e poi riempita con i file presenti nella cartella sorgente, se invece la cartella esiste allora viene creata in essa una directory che porta lo stesso nome della directory sorgente oltre poi a tutti i file in essa. Un caso particolare di utilizzo del comando è quando viene dato come percorso al file sorgente un file già esistente, quello che accade è che il file già esistente viene sovrascritto. Sotto riporto i classici esempi, nel primo utilizzo il comando copia più file in una directory, nel secondo si effettua una copia di una directory in un'altra già esistente e nel terzo si copia una directory in un'altra non esistente nel sistema, anche qui si parte dalla struttura di base riportata in 1.1:

Listato 1.12: cp

```

$ cd ./musica
$ cp fileA1 fileA2 preferiti
$ ls . ../preferiti
.:
fileA1      fileA2      fileA3      nuovi      preferiti
../preferiti:
fileA1      fileA2      fileC1      fileC2
$ cp -r ../preferiti ../scuola
$ ls ../scuola ../scuola/preferiti
../scuola:
preferiti
../scuola/preferiti:
fileA1      fileA2      fileC1      fileC2
$ cp -r nuovi ../varie
$ ls nuovi ../varie
../varie:
fileB1      fileB2
nuovi:
fileB1      fileB2

```

1.4.3. Il Globbing

Molto spesso riferirsi ad un file di sistema specificandone il nome esatto è abbastanza scomodo perchè richiede di ricordare non solo il nome del file ma anche il nome di tutte le sottodirectory che lo contengono, in pratica il percorso al file, il **globbing** è un linguaggio riconosciuto dalla shell che permette a l'utente di riferirsi a gruppi generici di file, per fare ciò viene usata una scrittura che permette all'utente di definire un pattern il quale viene utilizzato dalla shell per selezionare i file di interesse. I simboli usati per definire i pattern dipendono dalla shell utilizzata tuttavia lo *standard POSIX.1-2017* che viene seguito dalla gran parte delle shell fissa i seguenti:

- *: corrisponde a qualsiasi numero e tipo di carattere.
- ?: corrisponde a un carattere di qualsiasi tipo.
- [: corrisponde ad una classe di caratteri. Qualche esempio di intervallo definibile:
 - [ahz]: solo i caratteri a, h e z.
 - [a-c]: solo i caratteri compresi tra a e c.
 - [^a-c]: qualsiasi carattere tranne quelli compresi tra a e c. Il carattere ^ è interpretato nell'intervallo come negazione di quanto segue.
 - [1-3a-c]: solo i caratteri compresi tra 1 e 3 e compresi tra a e c.
 - [1-45-9]: solo i caratteri compresi tra 1 e 4 e quelli compresi tra 5 e 9. In particolare questo esempio permette di far notare un aspetto importante ovvero che, **quando si usa il carattere - per definire un range, la shell prende come limiti i singoli caratteri che si trovano a destra e sinistra di -**.

Gli esempi di intervalli sopra riportati richiedono di specificare a mano ogni volta il range di caratteri, un modo **avanzato per fare ciò evitando di scrivere i caratteri in particolare è usare le classi di caratteri, ne esistono di vari tipi e possono essere utilizzate per scrivere intervalli in maniera sintetica.**

Quando vengono usati i caratteri globbing la shell prima risolve questi e poi esegue il comando, questo comportamento va tenuto in considerazione perchè alcuni comandi potrebbero presentare un comportamento leggermente diverso da quello che ci si aspetterebbe, ad esempio il comando **ls** che quando l'argomento contiene caratteri globbing mostra in output i file che corrispondono al pattern definito. Come al solito riporto qualche esempio appena sotto.

Listato 1.13: Shell globbing

```
$ ls
file12ac file2v00 file2v02 file99jh fileX100 filef32 fileZAac
$ ls file2???
file2v00 file2v02
$ ls fileX*
fileX100
$ ls file[1-2]v0*
file2v00 file2v02
$ ls file[Y-Z]*
fileZAac
$ ls file[3-9A-Za-z]*
file99jh fileX100 fileZAac filef32
```

1.4.4. Ricerca di dati in un file

Avendo dei file è fondamentale riuscire a individuarli effettuando ricerche basate sul contenuto di essi, per fare ciò si usa il comando

`grep <flag> <pattern> <percorso_start_directory>`, in pratica il programma permette di cercare nei file contenuti nella directory passata (in realtà funziona anche se vengono passati singoli file, basta elencarli, tuttavia l'uso comune è su directory) le stringhe corrispondenti al pattern specificato, l'output consiste nel mostrare la riga dei file che contengono il dato in particolare che rispetta il pattern definito, il comando supporta molti flag, i più comuni sono riportati di seguito:

- -i: la ricerca non fa distinzione tra lettere minuscole e maiuscole.

- **-r**: la ricerca è ricorsiva, in pratica si applica anche al contenuto di file contenuti in sottodirectory.
- **-c**: la ricerca conta il numero di corrispondenze trovate.
- **-v**: inverte la ricerca, restituisce le righe che non hanno corrispondenza con i termini di ricerca.
- **-E**: attiva le espressioni regolari estese necessarie per scrivere pattern avanzati che usano i simboli |, +, ?.

Per definire i pattern di ricerca il comando prevede specifici simboli da usare:

- **^**: quanto segue deve essere presente all'inizio della riga.
- **\$**: quanto segue deve essere presente alla fine della riga.
- **.**: una sola occorrenza di qualsiasi carattere.
- *****: zero o più occorrenze del carattere precedente.
- **[]**: classe di caratteri accettati, è possibile esprimere intervalli di caratteri usando il simbolo -.
- **[^]**: negazione della classe di caratteri, sono accettati tutti quelli non presenti nell'elenco scritto.
- **+**: Una o più occorrenze del carattere precedente, tale simbolo è interpretabile solo se il comando ha il flag **-E**.
- **?**: Zero o una sola occorrenza del carattere precedente, tale simbolo è interpretabile solo se il comando ha il flag **-E**.
- **|**: OR logico, tale simbolo è interpretabile solo se il comando ha il flag **-E**.

Come si può vedere **il pattern utilizza caratteri speciali che la stessa shell potrebbe interpretare, per impedire ciò è buona norma scrivere sempre il pattern tra doppie virgolette ("")**. Di seguito si riportano vari esempi di utilizzo, questa volta il punto di partenza dell'esercizio è dato dal listing 1.14

Listato 1.14: Contenuto dei file presenti nella directory doc

```
$ cat file1
apple banana cherry
42 is the answer
grep is powerful
hello world 123
$ cat file2
john.doe@example.com
jane_smith1990@test.org
2024-03-03
03/03/202
$ cat file3
[ERROR] Failed to load module
[INFO] User logged in
[WARNING] Low disk space
[ERROR] Connection timeout
```

Ecco i vari esempi:

Listato 1.15: grep

```
$ grep "le" ./doc/*
./doc/file1:apple banana cherry
./doc/file2:john.doe@example.com
./doc/file3:[ERROR] Failed to load module
$ grep "le$" ./doc/*
./doc/file3:[ERROR] Failed to load module
$ grep "le " ./doc/*
./doc/file1:apple banana cherry
$ grep -E "apple|world" ./doc/*
./doc/file1:apple banana cherry
./doc/file1:hello world 123
$ grep "^U" ./doc/*
$ grep ".*@.*" ./doc/*
./doc/file2:john.doe@example.com
./doc/file2:jane_smith1990@test.org
$ grep ".*@.*\.com" ./doc/*
```

```
./doc/file2:john.doe@example.com
$ grep "^\[.*\]" ./doc/*
./doc/file3:[ERROR] Failed to load module
./doc/file3:[INFO] User logged in
./doc/file3:[WARNING] Low disk space
./doc/file3:[ERROR] Connection timeout
$ grep -E "2[0-9]?" ./doc/*
./doc/file1:42 is the answer
./doc/file1:hello world 123
./doc/file2:2024-03-03
./doc/file2:03/03/2024
$ grep -E "2[0-9]+" ./doc/*
./doc/file1:hello world 123
./doc/file2:2024-03-03
./doc/file2:03/03/2024
```

1.5. Compressione e Archiviazione dei file

Solitamente compressione e archiviazione vanno di pari passo tuttavia nei sistemi Linux non è detto che debba essere così e infatti le varie distribuzioni mettono a disposizione strumenti appositi per la compressione e strumenti appositi per l'archiviazione. Prima di vedere gli strumenti preosti alla compressione e archiviazione è bene ricordare che cosa si intende per compressione e archiviazione. La **compressione consiste in una operazione che permette di rendere più compatto un file in termini di occupazione di memoria non rinunciando però al contenuto informativo dello stesso**, questa operazione è possibile grazie all'utilizzo di particolari algoritmi che ricercano nel file originale pattern complessi che possono essere memorizzati in modo compatto e che permettono di ricostruire il contenuto informativo del file iniziale, bisogna aggiungere anche che **esistono algoritmi di compressione senza perdita, detti *lossless*, che praticamente nel comprimere il file fanno attenzione a non perdere nessun dato del file originale, e algoritmi di compressione con perdita, detti *lossy*, questi algoritmi nel comprimere il file, al fine di ottenere una riduzione ancora più significativa in termini di occupazione in memoria del file, tolgono dati "secondari"** (esempio tipico sono gli algoritmi di compressione per immagini che per diminuire il peso del file rimuovono i dati associati ad alcune sfumature che l'occhio umano non riesce a percepire oppure anche i metadati come luogo proprietario dell'immagine e tanti altri quindi, anche se praticamente si perdono dati, andando a decomprimere il file comunque la loro mancanza è impercettibile e quindi si dice che il contenuto informativo è preservato). Per quanto riguarda l'archiviazione, questa operazione consiste nel **raccogliere in modo ordinato più file (directory comprese) in un unico file generale**, questo processo non richiede necessariamente che i file al suo interno debbano essere compressi, la differenza rispetto ad una semplice directory è che l'archivio è un file contenente i dati ordinati estratti dai file usati per la sua costruzione, tutti questi dati si trovano praticamente in un unico file e questo aspetto è particolarmente comodo quando si vuole preservare l'ordine e la natura dei dati, ad esempio quando si devono spostare grandi quantità di file.

1.5.1. Strumenti per la compressione di file

I programmi di compressione si distinguono per il **tipo di algoritmo usato per comprimere i file e la bontà della compressione intesa come diminuzione delle dimensioni del file**, dipende principalmente dal tipo di dati presenti nel file e dalla velocità con cui si vuole effettuare l'operazione di compressione, infatti in molte distribuzioni Linux sono presenti più programmi per svolgere tale operazione, di seguito sono riportati i più comuni:

- **gzip**: usa algoritmo *lossless*, molto rapido con un rapporto di compressione basso, solitamente tra 60-70%, è utilizzato per comprimere file di log o dati temporanei.
- **bzip2**: usa algoritmo *lossless*, medio-lento ma con un rapporto di compressione buono 70-80%, è usato per archiviare generici file il cui accesso non deve avvenire in modo consueto.
- **xz**: usa algoritmo *lossless*, lento ma permette di ottenere compressione massima che solitamente consiste in un rapporto di compressione tra 80-90%, è impiegato per comprimere grandi archivi o anche intere distribuzioni.

Ciascun programma è invocabile da CLI attraverso apposito comando, ciascun comando ha struttura identica del tipo `<nome_programma> <flag> <percorso_file_da_comprimere>`, ogni comando ha un suo set di flag riconosciuti di seguito vengono riportati quelli comuni a tutti che sono poi anche i più frequentemente utilizzati

- `-d`: per indicare la decompressione, quindi il file passato deve essere decompresso.
- `-k`: il comando esegue compressione sul file passato ma ne lascia una copia.
- `-[1-9]`: indica il livello di compressione da applicare al file, 1 è il minimo quindi poca compressione ma rapida, 9 è il massimo quindi massima compressione però anche maggiore tempo per effettuarla.

Per tutti i programmi vale che il **file prodotto dalla compressione ha un nome formato dal nome originale del file più un suffisso particolare che ogni programma applica**, ad esempio `gzip` applica `.gz`, `bzip2` applica `.bz2` e `xz` applica `.xz`, tale funzionamento è molto importante per l'utente che può decomprimere il file sono ricordando con quale programma questo è stato compresso. Un file compresso resta ancora parzialmente consultabile, soprattutto da parte dell'utente, spesso infatti i programmi di compressione mettono a disposizione delle versioni speciali degli strumenti più comuni utilizzati per leggere i file di testo, per esempio `gzip` ha una versione di `cat`, `grep`, `diff`, `less`, `more` e alcuni altri e questi comandi sono semplicemente preceduti dal carattere `z`, per `bzip2` il prefisso è `bz` e per `xz` il prefisso è `xz`. Di seguito si riporta un esempio di utilizzo.

Listato 1.16: `gzip`, `zcat`, `bzip2`, `xz`

```
$ ls
file1 file2 file3 file4 file5
$ gzip file1 file2
$ bzip2 -5 file3
$ xz -lk file4 file5
$ ls
file1.gz file2.gz file3.bz2 file4.xz file5.xz file4 file5
$ bzip2 -d *.bz2
$ ls
file1.gz file2.gz file3 file4.xz file5.xz file4 file5
$ zcat file1.gz
ciao a tutti
```

1.5.2. Strumenti per l'archiviazione dei file

Il programma `tar` (abbreviativo di *tape archive* ovvero archivio su nastro) è probabilmente lo strumento di archiviazione più utilizzato sui sistemi Linux, i file creati con tale programma sono chiamati *tar ball*, il comando per eseguire il programma supporta molti flag, di seguito si riportano i più importanti e comunemente utilizzati:

- `-c`: crea un archivio.
- `-x`: estrae un archivio.
- `-v`: mostra dettagli durante l'operazione.
- `-t`: mostra il contenuto dell'archivio.
- `-f`: specifica il nome dell'archivio, se presente deve trovarsi sempre ultimo tra i flag in modo tale che la stringa successivamente digitata sia interpretata come il nome dell'archivio su cui si vuole operare.
- `-z`: comprime l'archivio con il programma `gzip`
- `-j`: comprime l'archivio con il programma `bzip2`
- `-J`: comprime l'archivio con il programma `xz`

I flag consentiti tendono a modificare anche la struttura del comando per questo non è stata riportata in chiaro, il **comportamento base del comando quando si desidera creare un archivio consiste nel copiare i dati dei file e directory specificate e poi scriverli nel file creato** (che è buona norma chiamare con un nome terminante con il suffisso `.tar` per il solito discorso che in questo modo sia immediatamente noto all'utente come deve maneggiare l'archivio). Creato un archivio **in genere l'operazione più comune che si vuole effettuare consiste nella lettura dei file contenuti in esso** (inteso non il contenuto dei file

ma il contenuto del file archivio che consiste in un elenco di file), a tale scopo si usa il flag **-t**, notare che il comando con tale flag mostra non solo i nomi dei file presenti nell'archivio ma anche il percorso relativo o assoluto di ciascuno nel momento in cui è stato inserito nell'archivio. L'operazione di estrazione dei file dall'archivio avviene inserendo il flag **-x**, il comportamento di default del comando è importante che sia ben chiaro: passato il file archivio al comando vengono creati tutti i file contenuti in esso, la posizione in cui ciascun file viene creato dipende dal percorso utilizzato durante la fase di inserimento di esso nell'archivio, pertanto se il file è stato inserito nell'archivio con un percorso assoluto allora, in fase di estrazione, il programma lo ricrea nel sistema seguendo precisamente il percorso assoluto specificato, se invece il file è stato inserito con percorso relativo allora viene seguito quello. In molti casi il comportamento di default del comando per l'estrazione è comodo tuttavia bisogna fare attenzione perchè se in fase di estrazione un file estratto possiede un nome identico a quello di un file già esistente (nella directory di destinazione) allora si verifica sovrascrittura (il file estratto sovrascrive quello preesistente). Come al solito sotto si riportano degli esempi, la struttura di partenza della directory user è quella mostrata in 1.1

Listato 1.17: tar

```
$ pwd
/home/user
$ tar -cf arc-doc.tar ./documenti
$ tar -cf ./documenti/arc-scuola.tar ./scuola
$ ls documenti .
documenti:
arc-scuola.tar fileZ1
.:
documenti musica scuola arc-doc.tar
$ tar -tf arc-doc.tar
./documenti/
./documenti/fileZ1
$ cp arc-doc.tar ./scuola
$ tar -xf arc-doc.tar
$ ls scuola
documenti
$ tar -czf arc-musica.tar.gz ./musica/fileA1 ./musica/fileA2
$ tar -tf arc-musica.tar.gz
./musica/
./musica/fileA1
./musica/fileA2
$ tar -xzf arc-musica.tar.gz
```

1.6. Redirezione di I/O

Normalmente la Command Line di Linux redirige le informazioni tramite specifici canali standard, la tastiera è considerata lo standard input (*stdin* o canale 0) di un comando e lo schermo è considerato lo standard output (*stdout* o canale 1), un altro canale ha lo scopo di redirigere l'output di errore (*stderr* o canale 2) di un comando o i messaggi di errore di un programma. Tuttavia in genere è molto utile redirigere l'input e/o l'output, ad esempio per passare ad un programma l'output generato da un altro oppure per avere l'output scritto su di un file anziché nella console, a tali scopi si usano programmi e comandi specifici. L'operatore *pipe* (**|**) è un particolare carattere che permette di creare le *pipeline* ovvero delle catene di comandi dove automaticamente l'output di un comando è usato come input dal successivo scritto dopo il carattere pipe, in altre parole risparmia l'operazione di reindirizzamento verso ad esempio file temporanei che poi dovrebbero essere letti da successivi comandi. Scrivendo un comando dove sono presenti più pipe i comandi intermedi sono chiamati *filtri* in quanto il loro effetto non è direttamente visibile all'utente essendo un comando il cui output è passato ad un altro comando.

1.6.1. Redirigere l' Output

Quando si desidera redirigere lo standard output (*stdout*, canale 1) di un comando in un file specifico si usano i simboli **>** e **>>**, il primo crea o sovrascrive il file dentro al quale riporta a modi testo l'output mentre il secondo crea o aggiunge al file specificato, a modi testo, l'output. Se si intende redirigere l'output che normalmente andrebbe sul canale 2 (*stderr*, output di errori) allora si devono usare i simboli **2 >** e **2>>**, il comportamento dei due è identico a quello descritto sopra. I simboli

& > e & > > reindirizzano sia il canale 1 che il canale 2 verso il file specificato dopo, anche qui la differenza tra i due è la solita, il primo sovrascrive o crea mentre il secondo aggiunge o crea. In tutte le distribuzioni Linux esiste il seguente file: /dev/null, questo file è considerato il *cestino* o *buco nero* e può essere utile reindirizzare gli output indesiderati in esso in quanto questo rimuove istantaneamente qualsiasi dato che riceve.

1.6.2. Redirigere l' Input

Come prima cosa è necessario capire cosa si intende per redirigere l'input, infatti il caso con l'output è immediato mentre quello con l'input richiede precisazioni; **Redirigere l'input è inteso come forzare l'utilizzo del canale 0 a sorgenti che normalmente non lo utilizzano, quindi ad esempio passare come argomento ad un comando il testo scritto in un file.** Il simbolo < è usato per passare come input tutto quello scritto nel file specificato, tipicamente è utilizzato con comandi che modificano le stringhe fornite in ingresso e mostrano sulla stdout il risultato, il simbolo << è usato per fornire input multiriga al comando, richiede che si definisca da subito un delimitatore che viene poi usato per delimitare il testo scritto.

1.7. Directory di sistema

Il sistema Unix è stato originariamente progettato per i computer mainframe a metà degli anni '60. Questi computer erano condivisi tra molti utenti che accedevano alle risorse di sistema tramite terminali. Questa idea di base è presente in moltissime distribuzioni Linux e tale aspetto si può notare osservando il modo in cui esso è organizzato in termini di directory di sistema, **tale modalità è diventata praticamente uno standard chiamata Filesystem Hierarchy Standard (FHS)**, la figura 1.2 mostra in cosa consiste tale standard in termini di directory.

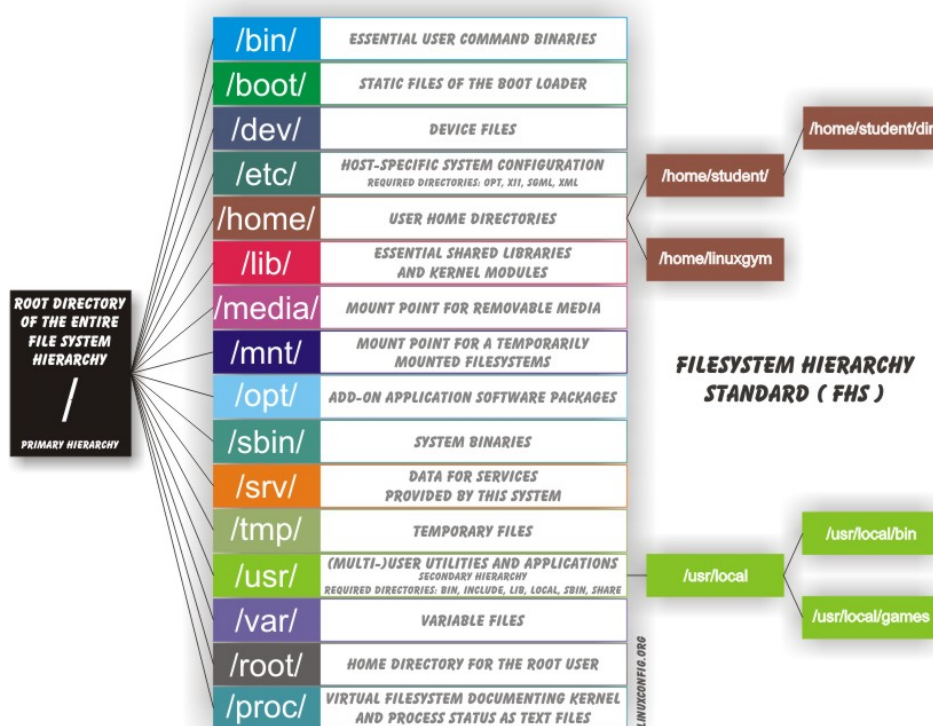


Figura 1.2: FHS

Il contenuto di queste cartelle riguarda l'intero sistema quindi tutti gli utenti e qualsiasi modifica ad essi richiede permessi amministratore. Nel seguito vengono spiegate le funzioni e i file che ogni cartella possiede.

1.7.1. La directory home

È una delle directory meno complesse, **contiene directory e file che ciascuno utente può liberamente creare e modificare senza dover richiedere particolari permessi root essendo i dati personali**. Tipicamente contiene sottodirectory di primo livello che riportano il nome dell'utente e in essa un numero variabile di altre sottodirectory e file completamente a discrezione dell'utente. Quando viene aperta una sessione shell, la CLI o il terminale, presenta un certo utente e la directory di lavoro iniziale è sempre `/home/nome_utente` abbreviata con il carattere `~`, se si vuole **cambiare utente e quindi sottodirectory nella directory home è necessario digitare il comando** `su <flag> <altro_utente>` i flag sono parecchi, per ora il più importante da ricordare è - il quale **permette di effettuare un cambio totale dell'utente dove infatti vengono importate anche le variabili di ambiente di altro_utente**, se non si utilizza tale flag si effettua un cambio parziale dove praticamente le variabili di ambiente usate dalla shell sono ancora quelle dell'utente iniziale. Quando si effettua un cambio totale dell'utente notiamo che il carattere `~` mantiene il suo significato ma il suo contenuto è diverso in quanto ora il percorso assoluto che rappresenta è `/home/altro_utente`. Quando si esegue il comando, **se l'utente a cui stiamo cercando di accedere possiede una password ovviamente in questa fase viene richiesta**. È possibile accedere a specifici file nelle cartelle di un altro utente senza dover effettuare il cambio di profilo anche se comunque questo richiederà la password. Un parallelismo efficace che descrive l'organizzazione della directory home è quello che la paragona ad un palazzo, ogni piano rappresenta un utente il quale è proprietario di tutto il piano e quindi può decidere liberamente come amministrarlo in termini di stanze (directory) e mobili (file), quanto decide un utente sul proprio piano non influenza quanto accade negli altri e se un utente vuole entrare nelle stanze altrui deve prima bussare alla porta del piano dell'altro utente (password del profilo). La manutenzione di tutto il palazzo è una responsabilità attribuita ad almeno uno degli utenti e (per dire che il buon funzionamento del sistema è garantito dall'amministratore che ha accesso a particolari file di configurazione)

1.7.2. La directory dev

È una directory fondamentale **all'interno della quale risiedono sottodirectory e file speciali che servono a interfacciare l'hardware montato con il sistema operativo**, possiamo dire che rappresentano il primo punto di contatto fra software e hardware tanto che viene gestita direttamente dal kernel che l'aggiorna dinamicamente. Nella directory dev sono presenti due tipi di file:

- *Character devices*: file creati tipicamente per dispositivi hardware che manipolano un carattere (quindi al massimo 1 o 2 byte) alla volta, tipicamente tastiere, mouse, terminali, porte seriali.
- *Block devices*: file creati tipicamente per dispositivi capaci di manipolare molti byte alla volta, ad esempio SSD, hard disk.

Molti dei file nella directory dev sono consultabili e modificabili dall'utente, va precisato che **modifiche a questi file hanno ripercussioni molto importanti sul sistema, se ad esempio si scrive direttamente e senza criterio nel file che interfaccia il sistema operativo con un dispositivo di memoria è molto probabile compromettere in modo irreversibile il dispositivo di memoria** che come minimo diventa inutilizzabile per il sistema operativo e, nei casi peggiori, tutti i dati in esso si corrompono. Molto più legittime sono le modifiche ai file driver dei dispositivi come tastiera, hub usb, monitor e altri che potrebbero richiedere all'utente di specificare particolari comportamenti tra sistema operativo e periferica. Bisogna sottolineare che in generale i file in questa directory, visto il significato che hanno per il sistema operativo, sono da modificare con molta attenzione, **però l'operazione di lettura non è mai dannosa per il file** (anche se potrebbe esserlo per il terminale o qualsiasi altro dispositivo usato per la lettura se l'output è molto grande e non limitato) **e se è noto il file particolare associato a del particolare hardware è possibile condurre indagini di debug molto efficienti**.

1.7.3. Le directory bin

Tutti i programmi eseguibili sul sistema Linux sono file binari e vengono memorizzati in varie directory di sistema che spesso portano nel nome il termine **bin**, esistono diverse directory bin perché il sistema segue tutta una serie di criteri per categorizzare i vari file binari:

- **/bin**: contiene i programmi essenziali per tutti gli utenti tipo `ls`, `cp`, `rm`, `mkdir`. Nelle distribuzioni più recenti non è presente ed è sostituita da `usr/bin`.

- **/sbin:** contiene programmi essenziali per l'amministrazione del sistema. come `fsck` e `textttreboot`. Nelle distribuzioni più recenti non è presente ed è sostituita da `usr/sbin`.
- **usr/bin:** contiene i programmi installati da utenti normali, tali programmi vengono scaricati, installati e aggiornati dal package manager della distribuzione.
- **usr/sbin:** contiene i programmi di amministrazione meno critici quindi in alcuni casi non necessari, anche questi sono scaricati, installati e aggiornati dal package manager.
- **usr/local/bin:** contiene programmi installati manualmente dagli utenti tipicamente si tratta di script bash scritti appositamente dall'utente per effettuare determinate operazioni sul sistema.
- **usr/local/sbin:** contiene strumenti amministrativi personalizzati quindi script o eseguibili personalizzati dall'amministratore e non gestiti dal package manager.
- **/opt:** contiene software proprietario, in pratica può contenere programmi non gestiti dal package manager e non direttamente collocabili per utilizzo in `usr/local/bin` e `usr/local/sbin`.

Si noti che l'utente **root** ovvero l'amministratore del sistema può consultare liberamente ciascuna delle directory sopra riportate e quindi può vedere quanti programmi sono installati sulla macchina indipendentemente dal numero di utenti che la utilizzano, invece un generico utente non può farlo a meno che l'amministratore non conceda i permessi necessari.

1.7.4. Le directory etc

La directory `/etc` contiene i file di configurazione di rete, sicurezza, applicazioni e utenti, tali file sono molto importanti perchè permettono di gestire il modo in cui il servizio viene fornito all'utente della macchina. In distribuzioni abbastanza datate i file di configurazione sono tipicamente lunghi file di testo, con il tempo si è visto che tale approccio non è comodo, soprattutto per quei servizi altamente configurabili, per questo, in distribuzioni recenti **tutti i file di configurazione inerenti ad un particolare servizio si trovano dentro una directory apposita, per mantenere una sorta di compatibilità le directory portano il nome originario del file e terminano con .d**. La directory internamente è molto articolata cioè sono presenti molte sottodirectory e file e non è facile darne una rappresentazione generica perchè molto dipende dal numero di programmi installati e in parte anche dal tipo di distribuzione in uso, di seguito si riportano le directory più importanti:

- **File di configurazione di sistema:**
 - **etc/passwd:** contiene l'elenco degli utenti con informazioni annesse tipo la home directory e la shell predefinita utilizzata.
 - **etc/shadow:** contiene le password cifrate degli utenti e gruppi e spesso anche la scadenza della password o l'ultima modifica.
 - **etc/group:** contiene l'elenco dei gruppi e dei loro membri, ad esempio il gruppo **sudo** è il classico gruppo amministratori, se un utente si trova in tale gruppo e conosce la password può operare come un amministratore.
 - **etc/fstab:** definisce il filesystem da montare all'avvio, senza scendere nei dettagli tecnici diciamo che le configurazioni in esso sono molto importanti per interfacciare il sistema con un dispositivo di memoria.
 - **etc/hostname:** contiene il nome della macchina visibile sulla rete.
- **File di configurazione di rete:**
 - **etc/hosts:** contiene un elenco che mappa gli indirizzi IP ai corrispettivi servizi.
 - **etc/resolv.conf:** contiene un elenco che mappa i servizi DNS.
 - **/etc/network/interfaces:** permette di gestire manualmente l'interfaccia di rete.
- **File di configurazione dei pacchetti:**
 - **/etc/apt/preferences.d/:** su sistemi Debian-based è una directory e contiene un file per ogni preferenza riguardando la gestione di un particolare pacchetto da parte del package manager, è molto utile quando si hanno molte repository da cui scaricare pacchetti e per il mantenimento di alcuni si preferisce scegliere una repository apposita.

- **/etc/apt/sources.list.d/**: su sistemi Debian-based è una directory e contiene un file per ogni indirizzo ad una repository che il gestore di pacchetti consulta per scaricare le applicazioni. Il modo in cui il gestore consulta queste repo dipende anche dai file di configurazione scritti in **/etc/apt/preferences.d/**.
- **File di configurazione sicurezza:**
 - **etc/sudoers**: specifica i permessi garantiti agli utenti presenti nel gruppo sudo.
 - **/etc/cron.d/**: permette di scrivere azioni pianificate che vengono eseguite secondo un certo criterio in modo automatico.

A livello utente i singoli programmi memorizzano le proprie configurazioni in file nascosti (il nome dei file iniziano con il carattere ".") presenti nella directory **/home/nome_utente**, distribuzioni recenti raccolgono tutti questi file in una directory di configurazione cioè: **/home/nome_utente/.config**.

2

Git

In questo capitolo viene trattato il software di controllo di versione *Git*, l'interesse verso questo strumento nasce sia dall'utilizzo di sistemi Linux, nei quali molti software vengono distribuiti tramite *repository Git*, sia dal fatto che Git rappresenta uno strumento fondamentale nello sviluppo software moderno.

Il capitolo è organizzato in sezioni e verrà progressivamente ampliato con l'introduzione di nuovi concetti seguendo un approccio incrementale basato sull'utilizzo diretto di Git.

2.1. Il concetto di versione in Git

Il *versioning* dei file è una pratica che permette di mantenere una cronologia delle modifiche rendendo possibile in qualsiasi momento risalire a uno stato precedente dei file. Un modo rudimentale per ottenere questo risultato consiste nel creare copie complete dell'intero archivio ogni volta che vengono effettuate delle modifiche e conservarle per eventuali ripristini futuri. Sebbene questo approccio possa sembrare inizialmente funzionale, presenta diverse limitazioni, in primo luogo la dimensione dell'archivio cresce rapidamente: più i file sono grandi, più le copie richiedono spazio di archiviazione, in secondo luogo, in un contesto condiviso tra più utenti, avere molte copie indipendenti (non collegate tra loro) rende difficile tracciare le modifiche effettuate e comprendere come il progetto si stia evolvendo nel tempo. Infine, anche la sincronizzazione su sistemi cloud risulta inefficiente, specialmente in presenza di archivi di grandi dimensioni.

Git nasce proprio per risolvere questi problemi introducendo un sistema di versionamento efficiente e strutturato. A differenza dei sistemi di controllo di versione centralizzati che si basano su un server remoto per gestire e sincronizzare le modifiche tra più utenti, **Git è un sistema distribuito ovvero l'intera cronologia del progetto è salvata in locale su ogni macchina**, l'indipendenza da un server che si trova in una qualsiasi rete rende estremamente più veloci le tipiche operazioni che un sistema di versionamento offre quindi, la possibilità di consultare la cronologia dei file, ripristinare versioni precedenti e proseguire con l'aggiornamento dell'archivio (aggiungere altre modifiche). Un altro aspetto fondamentale riguarda il modo in cui Git memorizza le modifiche infatti le dimensioni di un archivio **Git rimangono generalmente contenute grazie a un sistema basato su istantanee (snapshots)**. Per comprendere questo concetto è utile fare un confronto con altri sistemi di versionamento in particolare quelli che usano un approccio basato sulle differenze dove sostanzialmente viene salvata una versione iniziale dei file e, successivamente, solo le modifiche (dette *diff*) rispetto alla versione precedente. L'approccio per istantanee usato da Git prevede invece che ogni volta che viene salvata una nuova versione (commit), il sistema registri un'istananea completa dello stato dei file in quel momento (per evitare sprechi di spazio, Git non duplica realmente tutti i file, se un file non è stato modificato viene semplicemente mantenuto un riferimento alla versione già esistente). Riassunto in 2 righe, **Git riesce a combinare i vantaggi di una rappresentazione completa dello stato del progetto con un utilizzo efficiente dello spazio**.

Iniziamo ora ad analizzare in modo più pratico come funziona Git e come utilizzarlo. Una cartella gestita da Git è facilmente riconoscibile dalla presenza della directory `.git` che contiene tutte le informazioni

necessarie al versionamento del progetto, inclusa la cronologia delle versioni. Accanto a questa cartella possono essere presenti altri file, essi **rappresentano lo stato corrente del progetto, ovvero l'ultima versione salvata (detta commit) su cui è possibile lavorare**. Questi file possono essere modificati, eliminati o aggiunti e una volta completate le modifiche, è **possibile registrare una nuova versione del progetto utilizzando un comando Git** (`git commit`, successivamente spiegato). La costruzione di una versione prevede che ogni file possa trovarsi in uno dei seguenti tre stati:

- **Committed:** il file è presente nell'ultima versione del progetto, più in generale lo *stato* del file (*stato* inteso come contenuto del file) è riconducibile ad una delle versioni del progetto.
- **Modified:** il file è stato modificato rispetto alla versione di partenza ma le modifiche non sono ancora state confermate per essere inserite nella nuova versione che verrà creata.
- **Staged:** il file è stato selezionato per essere incluso nella prossima versione. In questo stato, Git "fotografa" lo stato corrente del file che costituirà la nuova versione.

Questi tre stati corrispondono a **tre aree logiche in cui è possibile suddividere il funzionamento di Git**:

- **Working Directory:** contiene i file su cui si lavora attivamente, volendo renderla concreta, rappresenta il contenuto in file della cartella.
- **Staging Area:** contiene, come istantanee, i file pronti per essere inclusi nella prossima versione.
- **Repository:** contiene, come istantanee, i file presenti nelle versioni salvate (commit) del progetto.

In termini operativi il flusso di lavoro tipico è il seguente: **si parte con l'avere tutti file committed appartenenti all'area Repository, eventualmente si modificano (passano allo stato modified) poi si selezionano le modifiche da includere nella nuova versione collocando lo stato dei file (stato inteso come contenuto) nella Staging Area, in quel frangente il file è nello stato staged, infine, con un commit, si salvano definitivamente tutte le istantanee dei file nella Staging Area nel Repository che di fatto diventano committed**, l'immagine nella figura 2.1 descrive il workflow appena descritto.

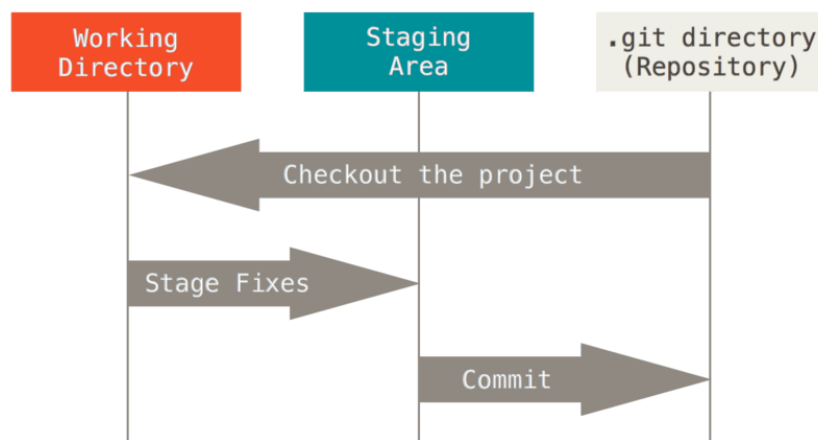


Figura 2.1: Tipico workflow git per effettuare il *versioning di file*

Dopo aver visto come funziona Git per quanto riguarda la creazione di una versione si passa a vedere come si fa in pratica, cioè a livello di comandi. Si può iniziare creando una cartella nel sistema, poi si apre Git Bash all'interno di essa e si digita il comando `git init`, **questo crea la cartella .git**, si può notare anche che compare la scritta `master` accanto al percorso, successivamente sarà maggiormente chiaro ma tale scritta identifica con precisione una particolare versione che è poi quella su cui si sta lavorando in termini di file. **I comandi** `git config -global user.name < nome >` e `git config -global user.email < email >` **servono per impostare l'identità di chi effettua i commit (quindi le versioni)**, sono semplici stringhe e non esiste un processo di autenticazione, l'opzione `-global` imposta come globali le configurazioni che si vanno a svolgere quindi, dopo questi comandi, qualsiasi cartella sottoposta a versionamento avrà versioni firmate con `< nome >` e `< email >` inseriti. Con il comando `git config -list` **si può vedere**

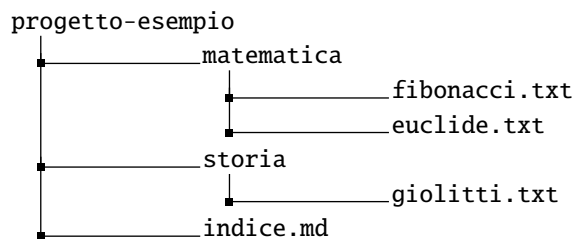
tutta la configurazione Git attiva per la particolare cartella, leggendo in corrispondenza della riga `core.editor= <nome editor attivo>` è possibile vedere quale editor interno Git utilizza, per cambiare "globalmente" questa impostazione si usa il comando `git config -global core.editor < nome editor >` dove `< nome editor >` deve essere tra quelli preinstallati nella versione di Git, tra i più comuni ci sono *nano* e *VIM*

La cartella sottoposta a versionamento, allo stato attuale, non contiene file effettivi del progetto (i file presenti in `.git` non rappresentano contenuto dell'archivio, ma sono utilizzati esclusivamente da Git per gestire il versionamento); è quindi possibile creare alcuni file di esempio. In questo caso si consideri la struttura riportata in figura 2.2, dove i file `.txt` contengono semplice testo, mentre il file `.md` presenta una struttura del tipo:

```
# matematica
- fibonacci
- euclide

# storia
- giolitti
```

Figura 2.2: Struttura della cartella progetto-esempio versionata con Git



A questo punto si eseguono i comandi `git add *` e successivamente `git commit -m "initial commit"`; la sequenza di questi due comandi crea la prima versione della cartella **progetto-esempio**, ovvero il **primo commit**, che per convenzione viene spesso chiamato *initial commit*. I file presenti nella cartella, visualizzabili ad esempio tramite il comando `ls`, risultano ora salvati nel repository e si trovano quindi nello stato *committed*. Si procede effettuando alcune modifiche, ad esempio si può modificare il file `euclide.txt` aggiungendo la parola "Teorema" e il file `fibonacci.txt` eliminando una parola qualsiasi, in seguito a queste operazioni i file passano dallo stato *committed* allo stato *modified*. Eseguendo il comando `git diff` è possibile visualizzare le differenze tra la versione salvata nel repository e quella attuale nella *working directory* (in realtà di default il comando `diff` confronta i file che si trovano nella staging area e *working directory*, tuttavia se la staging area non viene modificata, e questo accade dopo ogni commit prima di eseguire il comando `git add`, allora questa contiene solo i file *committed*), a seguire è riportato l'output per questo esempio:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git diff
warning: in the working copy of 'matematica/euclide.txt', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'matematica/fibonacci.txt', LF will be replaced by CRLF
the next time Git touches it
diff --git a/matematica/euclide.txt b/matematica/euclide.txt
index fd8eea5..c46511f 100644
--- a/matematica/euclide.txt
+++ b/matematica/euclide.txt
@@ -1,2 @@
Euclide padre della geometria.
+Teorema
diff --git a/matematica/fibonacci.txt b/matematica/fibonacci.txt
index 5f8a94b..ebd3924 100644
--- a/matematica/fibonacci.txt
+++ b/matematica/fibonacci.txt
@@ -1,1 @@
-Fibonacci matematico Pisano. La successione aurea.
+Fibonacci matematico. La successione aurea.
```

Sono presenti varie informazioni tecniche come le righe diff `-git ...`, `index ...`, ecc.), che servono a identificare i file e le versioni coinvolte nel confronto, ma che possono essere inizialmente trascurate, gli elementi fondamentali sono invece le righe precedute dai simboli:

- -, che indica una riga presente nella versione precedente ma rimossa
- +, che indica una riga aggiunta nella versione modificata

Nel caso dell'esempio, si osserva che nel file `euclide.txt` è stata aggiunta la riga Teorema, mentre nel file `fibonacci.txt` è stata modificata una riga tramite la rimozione di una parola (infatti nella versione attuale manca "Pisano"). Oltre a rilevare le modifiche, **Git segnala che esse non sono ancora state incluse nella prossima versione; infatti, eseguendo il comando `git status -u`, si ottiene un output che indica chiaramente come i file modificati non siano ancora stati aggiunti alla staging area**, output del comando a seguire:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
modified:   matematica/euclide.txt
modified:   matematica/fibonacci.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Git evidenzia che i file `euclide.txt` e `fibonacci.txt` sono nello stato *modified*, ma non ancora nello stato *staged*, e quindi non verranno inclusi nel prossimo commit. Se si desidera **includere nel prossimo commit lo stato attuale del file `euclide.txt`**, è possibile utilizzare il comando `git add matematica/euclide.txt`; in questo modo il file passa allo stato *staged*. Eseguendo nuovamente `git status`, si osserva che tale file è ora pronto per essere incluso nella nuova versione, mentre gli altri file rimangono invariati rispetto all'ultimo commit. **Si può quindi eseguire il comando `git commit -m "modificato il file euclide"`, creando una nuova versione del progetto.** A questo punto, la **working directory** contiene sia file aggiornati all'ultima versione (*committed*), sia file che presentano modifiche non ancora salvate, come nel caso di `fibonacci.txt`. Utilizzando nuovamente `git diff`, si nota che le differenze relative a `fibonacci.txt` sono le stesse di prima, poiché il confronto avviene sempre rispetto all'ultima versione salvata nel repository (che è la stessa della prima versione). Può sembrare quindi che un file sia contemporaneamente *committed* e *modified*, ma **in realtà si stanno considerando due versioni diverse dello stesso file: quella salvata nel repository e quella attualmente modificata nella working directory.** Si può proseguire con l'esempio aggiungendo il file `fibonacci.txt` alla Staging area quindi si esegue `git add matematica/fibonacci.txt`, a questo punto si può provare a modificare ancora una volta il file `fibonacci.txt`, si aggiunge la parola "Spirale", adesso eseguendo il comando `git status`:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
modified:   matematica/fibonacci.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
modified:   matematica/fibonacci.txt
```

si vede come lo stesso file risulti sia *modified* che *staged* ma ancora una volta non è così perchè quelli che si stanno considerando sono per Git due file diversi essendo che hanno un contenuto diverso. Facendo `git diff` ora si vedono chiaramente le differenze tra la versione del file *staged* e la versione del file *modified* cioè quello nella working directory, con il comando `git diff --staged` invece si vedono le differenze tra la versione del file *committed* (quello nell'area repository) e la versione del file *modified*, ancora, quello nella working directory, per completezza lascio sotto i due output.

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
```

```
$ git diff
warning: in the working copy of 'matematica/fibonacci.txt', LF will be replaced by CRLF
the next time Git touches it
diff --git a/matematica/fibonacci.txt b/matematica/fibonacci.txt
index ebd3924..3bf46c9 100644
--- a/matematica/fibonacci.txt
+++ b/matematica/fibonacci.txt
@@ -1,1 @@
-Fibonacci matematico. La successione aurea.
+Fibonacci matematico. La successione aurea. Spirale
```

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git diff --staged
diff --git a/matematica/fibonacci.txt b/matematica/fibonacci.txt
index 5f8a94b..ebd3924 100644
--- a/matematica/fibonacci.txt
+++ b/matematica/fibonacci.txt
@@ -1,1 @@
-Fibonacci matematico Pisano. La successione aurea.
+Fibonacci matematico. La successione aurea.
```

Si procedere effettuando un commit, si può facilmente verificare che la versione divenuta *committed* del file `fibonacci.txt` è quella che prima era stata inserita nella staging area. Quest'ultimo esempio è utile per far capire che *Git tratta i file come istantanee ovvero quando si aggiungono file alla staging area è come se questi venissero fotografati e tale fotografia sarà presente nel successivo commit che si può allora intendere come una raccolta di foto.*

Con queste nozioni è possibile solo proseguire in avanti ovvero si possono continuare a creare versioni della cartella usando Git ma senza beneficiare di nessun *vantaggio*.

2.2. I Branch

Per comprendere il funzionamento dei branch in Git è utile introdurre una rappresentazione grafica della cronologia dei commit; a tal fine, il comando `git log --all --oneline --graph` consente di visualizzare la sequenza delle versioni sotto forma di una struttura ad albero mettendo in evidenza come i commit siano collegati tra loro. A seguire è riportato un output di esempio in cui praticamente tutte le versioni del progetto sono, per ora, allineate su un unico ramo.

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git log --all --oneline --graph
* 8f32569 (HEAD -> master) modificato il file giolitti
* 2877e77 modificato il file giolitti
* 987e4ba aggiornato file fibonacci
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
* dfcd8f9 initial commit
```

Si può osservare che ogni *commit* è identificato in modo univoco da un hash, utilizzando l'opzione `--oneline` vengono mostrate solo le prime 7 cifre, ma l'identificativo completo è lungo 40 caratteri. Questo hash viene generato automaticamente da Git in base al contenuto del progetto e garantisce che ogni versione sia univoca, rendendo impossibile modificare retroattivamente un commit senza generarne uno nuovo. Nell'output compare anche il riferimento `HEAD` che rappresenta un puntatore al commit attualmente selezionato, ovvero alla versione del progetto su cui si sta lavorando; nel caso mostrato, `HEAD` coincide con il branch `master` e punta all'ultimo commit. Utilizzando il comando `git checkout <hash commit>` è possibile spostare `HEAD` su un commit specifico rendendo quella versione la base di lavoro corrente e, di conseguenza, i file presenti nella *working directory* corrisponderanno allo stato salvato in quel commit. Una nota importante è che in genere i file *modified* vengono trasportati da una versione all'altra e questo è un problema perché si rischia di mischiare file appartenenti a versioni diverse, per evitare questa situazione si può usare il comando `git stash -u` che praticamente crea una copia di tutti i file nella *working directory* e nella *staging area* e li mette in uno stack di tipo LIFO (Last In, First Out), questo permette di cambiare versione avendo le zone di lavoro pulite. Per ripristinare l'ultimo stato memorizzato nello stack si usa `git stash pop`, anche qui è importante precisare che il ripristino sarebbe meglio effettuarlo avendo

working directory e **staging area** vuote e anche in corrispondenza dello stesso commit in cui era stato eseguito il salvataggio dello stato. Oltre a questo utilizzo, `git stash` è usato anche per altre operazioni come ad esempio trasportare file modificati da un commit ad un altro (per fare confronti o qualsiasi altra operazione), a tal fine tornano utili i seguenti comandi: `git stash list`, che mostra tutti i salvataggi presenti nello stack (ogni salvataggio è indicizzato con un indice assegnato in modo progressivo), `git stash apply <indice>`, che ripristina uno specifico salvataggio nella posizione corrente e senza rimuoverlo dallo stack, e `git stash drop <indice>`, che elimina un elemento dallo stack.

A questo punto, essendo possibile spostarsi tra diverse versioni, si può osservare un comportamento fondamentale di Git. **Se si seleziona un commit precedente, si modificano i file e si crea un nuovo commit, la cronologia non prosegue semplicemente in avanti, ma si genera una nuova linea di sviluppo parallela.** Si consideri il seguente output, ottenuto dopo aver effettuato un commit a partire dal 529e5f6:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti ((5a0319a...))
$ git log --all --oneline --graph
* 5a0319a (HEAD) aggiunto aroldi a storia
| * 8f32569 (master) modificato il file giolitti
| * 2877e77 modificato il file giolitti
| * 987e4ba aggiornato file fibonacci
|/
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
* dfcd8f9 initial commit
```

In questo caso si osserva chiaramente la presenza di due rami: uno principale, identificato dal puntatore `master`, e uno secondario, su cui si trova attualmente `HEAD`. Il nuovo commit non è stato aggiunto in coda alla linea principale, ma ha dato origine a un nuovo percorso nella cronologia, **questo accade perché ogni commit contiene un riferimento al commit precedente e creando un commit a partire da una versione passata si costruisce una nuova sequenza indipendente che si sviluppa parallelamente a quella già esistente.** Si nota inoltre che `HEAD` non punta più a `master`, ma direttamente a un commit, questa situazione è nota come *detached HEAD* e indica che non si sta lavorando su un branch, ma su un commit specifico. Per rendere questa **nuova linea di sviluppo persistente e facilmente accessibile (altrimenti sarebbe necessario usare ogni volta hash dell'ultimo commit)** si crea un branch usando il comando `git branch <nome del branch>`, una volta creato è possibile spostarsi su di esso e continuare lo sviluppo in modo strutturato mantenendo separate le diverse linee di evoluzione del progetto. A seguire output di `git log --all --oneline --graph` dopo aver "creato" il nuovo branch e aver spostato `HEAD` su esso.

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (mattia)
$ git log --all --oneline --graph
* 5a0319a (HEAD -> mattia) aggiunto aroldi a storia
| * 8f32569 (master) modificato il file giolitti
| * 2877e77 modificato il file giolitti
| * 987e4ba aggiornato file fibonacci
|/
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
* dfcd8f9 initial commit
```

Di default Git crea sempre il ramo `main` (chiamato anche `master` in alcune versioni recenti) e una buona regola è mantenere tale branch come linea di sviluppo principale del progetto, ovvero tutti gli eventuali branch ad un certo punto devono confluire in esso.

2.3. Merge

Dopo aver introdotto il concetto di branch, è possibile analizzare **come e perché questi possano essere uniti, tale operazione prende il nome di merge tra branch.** Nella maggior parte dei casi il merge viene utilizzato per integrare un branch secondario (ad esempio sviluppato per una nuova funzionalità) all'interno del branch principale. Il comando di base per effettuare un merge è `git merge <branch>`: questa operazione integra i contenuti del branch specificato all'interno del branch corrente (ricordando

che il nome di un branch rappresenta un puntatore all'ultimo commit di una determinata linea di sviluppo). L'integrazione può avvenire in due modi:

- **Diretta:** quando le modifiche riguardano parti diverse dei file, il merge viene eseguito automaticamente senza richiedere intervento.
- **Conflittuale:** quando le modifiche interessano le stesse parti di uno o più file, è necessario un intervento manuale.

Il caso diretto è immediato, quello conflittuale richiede qualche attenzione in più, infatti, **quando due branch modificano un file nello stesso punto (ad esempio la stessa riga), Git non è in grado di stabilire quale versione mantenere e richiede quindi una risoluzione manuale.** In questi casi **Git evidenzia il conflitto direttamente all'interno dei file** usando marcatori speciali che delimitano le diverse versioni in conflitto e **l'utente deve modificare il file scegliendo cosa mantenere (una delle due versioni oppure una combinazione di entrambe), salvare il risultato e aggiungere questi alla staging area, poi effettuare un commit che rappresenta il completamento del merge;** i file non coinvolti nei conflitti vengono invece integrati automaticamente.

Un esempio che mostra entrambe le situazioni. L'albero di partenza è il seguente:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (giorgia)
$ git log --all --oneline --graph
* ffbb618 (HEAD -> giorgia) aggiuta cartella musica e Verdi
| * 33f6bce (master) modificato il file euclide
| * 8f32569 modificato il file giolitti
| * 2877e77 modificato il file giolitti
| * 987e4ba aggiornato file fibonacci
|/
| * 1ab128a (mattia) modificato file euclide
| * 5a0319a aggiunto aroldi a storia
|/
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
* dfcd8f9 initial commit
```

Sono presenti tre branch: uno principale, master, e due secondari, mattia e giorgia. Il contenuto del file euclide.txt nei vari branch è il seguente:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (mattia)
$ cat matematica/euclide.txt
Euclide padre della geometria.
Teorema
Atene.
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (mattia)
$ git checkout master
Switched to branch 'master'
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ cat matematica/euclide.txt
Euclide padre della geometria.
Teorema
Platone
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (mattia)
$ git checkout giorgia
Switched to branch 'giorgia'
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (giorgia)
$ cat matematica/euclide.txt
Euclide padre della geometria.
Teorema
```

Nel branch giorgia è inoltre presente una cartella aggiuntiva musica contenente Verdi.txt. Si tenta ora un merge tra master e mattia, è naturale aspettarsi un conflitto sul file euclide.txt in quanto modificato nello stesso punto:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git merge mattia
Auto-merging matematica/euclide.txt
CONFLICT (content): Merge conflict in matematica/euclide.txt
```

```
Automatic merge failed; fix conflicts and then commit the result.
```

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master|MERGING)
```

durante questa fase, il repository si trova in uno stato intermedio indicato come (master|MERGING), che segnala la presenza di un merge non ancora completato, il file in conflitto appare nel seguente modo:

```
<<<<<< HEAD
Platone
=====
Atene.
>>>>>> mattia
```

i marcatori indicano:

- HEAD: versione del branch corrente
- mattia: versione del branch da integrare

Si può quindi risolvere il conflitto mantenendo entrambe le versioni, modificando il file e rimuovendo i marcatori:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master|MERGING)
$ cat matematica/euclide.txt
Euclide padre della geometria.
Teorema.
Platone.
Atene.
```

si eseguono `git add *` e `git commit`, completando il merge. Il risultato è un nuovo commit che unisce le due linee di sviluppo. Successivamente, si può effettuare il merge del branch giorgia in master; in questo caso non si verificano conflitti, poiché le modifiche riguardano file diversi o non sovrapposti (in termini di contenuto):

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git merge giorgia
Merge made by the 'ort' strategy.
musica/Verdi.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 musica/Verdi.txt
```

in questo caso Git esegue automaticamente l'integrazione e crea direttamente un commit di merge (eventualmente aprendo l'editor per inserire il titolo del commit). Al termine delle operazioni, il grafo dei commit mostra chiaramente l'unione delle diverse linee di sviluppo:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/appunti (master)
$ git log --all --oneline --graph
* 30d99d9 (HEAD -> master) merge master-giorgia
|\
| * ffbb618 (giorgia) aggiusta cartella musica e Verdi
* | 3183e7f fix merge master-mattia
|\ \
| * | 1ab128a (mattia) modificato file euclide
| * | 5a0319a aggiunto aroldi a storia
| | /
* | 33f6bce modificato il file euclide
* | 8f32569 modificato il file giolitti
* | 2877e77 modificato il file giolitti
* | 987e4ba aggiornato file fibonacci
| /
* 529e5f6 modificato il file euclide
* 6ae82da modificati i file in matematica
* dfcd8f9 initial commit
```

L'unico vero branch rimasto è il master, gli altri due continuano ad esistere, infatti eseguendo il comando `git branch --list` compaiono nella lista, tuttavia solitamente si eliminano essendo che i branch non dovrebbero essere più sviluppabili, per fare ciò si usa il comando `git branch -d <branch>`.

3

GitHub

In questo capitolo viene trattata la piattaforma *GitHub* e, più in particolare, la sua **integrazione con il software Git, questo binomio è oggi ampiamente diffuso nello sviluppo software**, soprattutto nel contesto open source. Come visto nel capitolo 2, Git è uno strumento per il versionamento dei file e fornisce meccanismi che si prestano naturalmente allo sviluppo collaborativo, tuttavia, **Git opera in modo completamente locale e non fornisce direttamente strumenti per la condivisione e la sincronizzazione dei repository tra più utenti**. In assenza di una piattaforma remota, la condivisione di un archivio richiederebbe infatti il trasferimento manuale dei dati tra macchine diverse (ad esempio tramite dispositivi fisici), rendendo difficile gestire in modo efficace il lavoro collaborativo.

GitHub nasce proprio per risolvere queste problematiche offrendo **un'infrastruttura online che consente di ospitare repository Git e di renderli accessibili tramite rete**. In questo modo la collaborazione tra più utenti diventa una caratteristica pienamente supportata grazie a strumenti che permettono la sincronizzazione delle modifiche, la gestione dei contributi e il coordinamento dello sviluppo.

3.1. Repository remoti

Con il termine *repository remoto* si indica, **in modo generale, un archivio di file che si trova su una macchina diversa rispetto a quella su cui si sta lavorando**. L'utilizzo di repository remoti risulta particolarmente utile in diversi scenari: da un lato **permette di condividere un progetto tra più macchine o utenti, dall'altro consente di mantenere una copia di sicurezza dell'archivio in una posizione separata**.

Nell'integrazione tra **Git e GitHub**, il repository remoto rappresenta una versione del repository Git ospitata su un server online, ovvero GitHub, e il suo scopo principale è quello di **permettere la collaborazione tra più utenti**. La condivisione, in questo contesto, non si limita alla semplice consultazione o al download dell'archivio, ma comprende anche la **possibilità di modificarlo e aggiornare la versione condivisa**. In termini operativi, il flusso di lavoro tipico prevede che un utente scarichi una copia del repository, apporti modifiche localmente utilizzando Git e successivamente invii tali modifiche al repository remoto, aggiornandone il contenuto. Sebbene questo processo possa apparire semplice, nella pratica emergono problematiche non banali, come la gestione di modifiche simultanee da parte di più utenti. Git e GitHub forniscono strumenti specifici per affrontare queste situazioni, evitando la perdita di dati e garantendo che le diverse versioni del progetto vengano integrate in modo coerente.

3.2. Creare e gestire repository remoti su GitHub

Per creare una repository remota su GitHub è sufficiente andare sulla pagina web di GitHub e creare un account, una volta nell'area personale basta premere il pulsante "New". Successivamente è possibile specificare il nome della repository e la tipologia, *public* o *private*: nel primo caso la repository è accessibile pubblicamente, mentre nel secondo è visibile e modificabile solo dagli utenti autorizzati, di default lo stesso che ha creato la repository. Una volta completata la creazione **GitHub fornisce un URL**

che identifica univocamente la repository e che rappresenta l'elemento principale per interagirla tramite Git. Sebbene GitHub metta a disposizione anche un editor web per creare, modificare ed eliminare file direttamente dal browser, è consigliabile limitarne l'utilizzo, soprattutto nelle fasi iniziali, in quanto può introdurre modifiche non sincronizzate con il repository locale e rendere meno chiaro il flusso di lavoro.

Per iniziare a lavorare sulla repository remota, è possibile clonarla in locale tramite il comando `git clone <URL> <cartella>`, questo crea una copia completa della repository remota all'interno di una cartella locale (il cui percorso è da specificare in <cartella>, percorso relativo rispetto alla posizione in cui si trova avviata la sessione di Git Bash); nel caso di repository private potrebbe essere richiesto di effettuare l'autenticazione prima di completare l'operazione. Dopo il clone, il repository locale è già configurato per comunicare con quello remoto, infatti Git crea automaticamente un riferimento chiamato *origin*, che memorizza l'URL della repository remota. Banalmente, avendo l'URL, Git permette, attraverso comandi specifici, di effettuare richieste web al server GitHub che ospita la repository remota. Per verificare i repository remoti associati, è possibile utilizzare il comando `git remote -v` (si possono configurare più repository remoti per uno stesso repository locale ma questa è un'applicazione abbastanza avanzata) che mostra gli alias configurati e i relativi URL (gli alias sarebbero i nomi associati agli URL), in questo caso di esempio si dovrebbe avere un output di questo tipo:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2
$ git remote -v
origin https://github.com/ViscidoSnake/esempio2.git (fetch)
origin https://github.com/ViscidoSnake/esempio2.git (push)
```

cioè un solo alias, *origin*, che Git utilizza per effettuare le operazioni di *push* e *fetch*, operazioni fondamentali per mantenere l'aggiornamento tra repository remoto e repository locale. In parole molto semplici l'operazione di *push* carica nella repository remota i commit avvenuti nella repository locale mentre il *fetch* serve per scaricare nella cartella locale i cambiamenti e riferimenti Git dalla repository remota verso quella locale. Le modalità con cui si possono caricare update avvenuti nella cartella locale sono molte e tutti attraverso il comando `git push <opzioni>`, il modo in assoluto più semplice è quello di aggiungere nuovi commit, senza quindi cambiare radicalmente la struttura di partenza del repository remoto (questo accade se si inizia ad usare per esempio `git reset`), per fare questo si usa il comando `git push <alias> <branch locale>` dove appunto alias rappresenta un URL che punta alla repository remota e branch sarebbe il nome del branch locale da caricare, questa operazione non carica semplicemente l'ultimo commit ma tutti gli ultimi commit avvenuti sul particolare branch. Volendo provare, una volta effettuato il clone, basta creare un file, generare il primo commit, e poi fare il push nella repository remota, aprendo GitHub dovrebbe comparire il nuovo file. Oltre a vedere il file è interessante guardare anche l'output del comando:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 221 bytes | 221.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/ViscidoSnake/esempio2.git
* [new branch]      main -> main
```

dopo le prime righe che indicano quanti file sono stati caricati e la dimensione di essi, l'ultima riga informa che è stato creato un nuovo branch, questa volta remoto, che porta lo stesso nome del branch appena *pushato* (in realtà, come poi si vede anche dal log, il nome del branch remoto creato ha come prefisso l'alias che la repository locale ha usato per il push, in questo caso *origin*) e in più i due sono perfettamente allineati cioè quindi le versioni dell'archivio locale e dell'archivio remoto sono identiche. Eseguendo `git log --all --branch` è possibile vedere graficamente quanto detto. Dunque la cosa importante è che la sincronizzazione è un'operazione affidata all'utente che deve appunto aggiornare progressivamente la repository remota dopo aver apportato le modifiche sulla repository locale, in caso contrario si creano versioni non sincronizzate e ciò lo si può vedere dal grafo generato da `git log --all --graph` dove il branch remoto resta indietro rispetto a quello locale, in riferimento all'esempio di prima:

```

snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (main)
$ git log --all --graph
* commit 4c8f7df447945c27939e3e9a2720ad04499958a1 (HEAD -> main)
| Author: Elia <Elia>
| Date: Thu Mar 26 09:32:14 2026 +0100
|
|      modificato file listaSpesa, aggiunte
|
* commit e194fd454fa71e2f2c111553acf8c37a993f2b06
| Author: Elia <Elia>
| Date: Thu Mar 26 09:30:07 2026 +0100
|
|      modificato file note, aggiunto pollo
|
* commit ce5262c27d169420db89519b0ffab163e0e8ce20
| Author: Elia <Elia>
| Date: Thu Mar 26 09:29:11 2026 +0100
|
|      aggiunto file note
|
* commit dfa0d0106007b2cd13181e664ecacc7cda569b84 (origin/main)
Author: Elia <Elia>
Date: Thu Mar 26 09:12:08 2026 +0100

initial commit

```

Un'altra questione importante riguarda lo **scaricamento dell'archivio modificato da GitHub verso la repository locale**, questo si rende necessario se, per vari motivi, la versione remota è maggiormente aggiornata rispetto a quella locale, in questi casi si utilizzano i comandi `git fetch --all` e `git pull <alias> <branch remoto>`. Partendo dal primo, questo scarica i file nella cartella `.git` e permette, dalla repository locale, di vedere la timeline aggiornata dell'archivio eseguendo il comando `git log --all --graph`, vedendo il grafo è possibile capire quali update effettivamente ha ricevuto la repository remota rispetto a quella locale, per applicare tali cambiamenti in termini di file si usa il comando `git pull`, questa operazione va fatta sempre con cautela perchè il **pull del particolare ramo remoto va a fondersi implicitamente con i file contenuti nel ramo locale in cui Git si trova**, è quindi fondamentale assicurarsi sempre di posizionarsi nello stesso branch da *riallineare* e anche avere un staging area e working directory pulite. Una cosa che può accadere è **vedere dal log la presenza di un branch remoto non presente nella versione attuale della repository locale, qui è impossibile effettuare un pull diretto perchè il branch locale non esiste**, si potrebbe crearlo a partire dal giusto commit (informazione che si legge dal grafo), spostarsi sul nuovo branch e poi fare il pull, tuttavia tutto questo può essere eseguito da un solo **comando ovvero** `git checkout -b <branch locale> <alias>/<branch remoto>`; di seguito un esempio del caso appena descritto, per simulare l'aggiunta di un branch sulla repository remota è stato usato l'editor di GitHub. Partendo dal grafo mostrato appena sopra, da GitHub è stato aggiunto un branch chiamato "marco" ed è stato aggiunto il file `festa.txt`, eseguendo ora il comando `git fetch`:

```

snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (main)
$ git fetch
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 988 bytes | 89.00 KiB/s, done.
From https://github.com/ViscidoSnake/esempio2
* [new branch]      marco      -> origin/marco

```

per vedere chiaramente quali nuovi branch sono presenti nella versione remota:

```

snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (main)
$ git log --all --graph
* commit 5320f73bd053088bbc0e50eaf20d62218220454c (origin/marco)
| Author: ViscidoSnake <132934376+ViscidoSnake@users.noreply.github.com>
| Date: Fri Mar 27 09:36:07 2026 +0100
|

```

```
|      creato il file festa.txt
|
* commit e194fd454fa71e2f2c111553acf8c37a993f2b06 (HEAD -> main, origin/main)
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:30:07 2026 +0100
|
|      modificato file note, aggiunto pollo
|
* commit ce5262c27d169420db89519b0ffab163e0e8ce20
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:29:11 2026 +0100
|
|      aggiunto file note
|
* commit dfa0d0106007b2cd13181e664ecacc7cda569b84
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:12:08 2026 +0100
|
initial commit
```

a questo punto allora eseguo il comando `git checkout -b marco origin/marco`, si può vedere che è stato creato un nuovo branch locale perfettamente allineato a quello remoto.

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (main)
$ git checkout -b marco "origin/marco"
Switched to a new branch 'marco'
branch 'marco' set up to track 'origin/marco'.

snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (marco)
$ git log --all --graph
* commit 5320f73bd053088bbc0e50eaf20d62218220454c (HEAD -> marco, origin/marco)
| Author: ViscidoSnake <132934376+ViscidoSnake@users.noreply.github.com>
| Date:   Fri Mar 27 09:36:07 2026 +0100
|
|      creato il file festa.txt
|
* commit e194fd454fa71e2f2c111553acf8c37a993f2b06 (origin/main, main)
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:30:07 2026 +0100
|
|      modificato file note, aggiunto pollo
|
* commit ce5262c27d169420db89519b0ffab163e0e8ce20
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:29:11 2026 +0100
|
|      aggiunto file note
|
* commit dfa0d0106007b2cd13181e664ecacc7cda569b84
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:12:08 2026 +0100
|
initial commit

snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (marco)
$ ll
total 3
-rw-r--r-- 1 snake 197121 17 Mar 27 09:46 festa.txt
-rw-r--r-- 1 snake 197121 26 Mar 27 09:26 listaSpesa.md
-rw-r--r-- 1 snake 197121 78 Mar 27 09:26 notal.txt
```

Va precisata una questione, **l'utilizzo della repository remota e il modo di gestirla sopra descritti non sono chiaramente gli unici modi possibili**, infatti non sono stati affrontati temi cruciali come che cosa accade se si modifica in modo diverso uno stesso commit e poi si effettua il push, oppure cosa accade se si eliminano branch dalla repository remota; per gestire queste dinamiche complesse è necessario approfondire lo studio dei comandi `git push` e `git pull` i quali supportano molte opzioni che ne modificano in modo radicale il comportamento. Oltre a questo in genere esistono delle regole

implicite nella gestione dell'archivio remoto che evitano proprio il verificarsi di situazioni complicate da sistemare, le più comuni sono:

- evitare di riscrivere la cronologia dei commit già condivisi (ad esempio tramite `git reset` seguito da push forzati), in quanto ciò può creare inconsistenze tra le copie locali dei vari collaboratori;
- evitare di lavorare direttamente sul branch principale, riservandolo a versioni stabili del progetto;
- effettuare aggiornamenti frequenti (pull) prima di eseguire un push, in modo da ridurre la probabilità di conflitti;
- utilizzare branch separati per funzionalità diverse, così da isolare le modifiche e semplificare eventuali merge.

Questo per **sottolineare che le modalità di gestione descritte nei paragrafi precedenti sono pensate per casi relativamente semplici, in cui l'obiettivo principale è mantenere la repository remota perfettamente allineata a quella locale.** In questo contesto, anche la collaborazione tra più utenti segue una logica ben definita: ciascun **collaboratore opera su un proprio branch, evitando che altri possano effettuare commit o operazioni di merge sugli stessi rami**, così da ridurre al minimo possibili interferenze e semplificare la gestione complessiva dell'archivio.

Fonti

- [1] Website Name <OR> I. Surname, I. Surname e I. Surname. *Title of the Website*. 2000. URL: <https://example.com> (visitato il 24/12/2020).